[UNIVERSITY NAME]

[FACULTY / DEPARTMENT]

CODE COMMENT CLASSIFICATION USING MACHINE LEARNING AND DEEP LEARNING

Research Report

Master Seminar 2

NLBSE'23 – Code Comment Classification

Course	Master Seminar 2
Instructor	[INSTRUCTOR]
Research group	[RESEARCH GROUP]
Authors	[AUTHOR NAMES AND STUDENT IDS]
Submission date	[SUBMISSION DATE]

English edition

ABSTRACT

This report investigates multi-label classification of software comment sentences using the NLBSE'23 corpus. The audited population contains 6,738 sentences from Python, Java, and Pharo, with five, seven, and seven language-specific labels. A shared preprocessing pipeline pivots category-wise binary records into complete sentence-level targets and keeps identical normalized texts in the same group. Seventeen RoBERTa and CodeBERT configurations are compared using five-fold screening on the entire population, followed by fixed 80/10/10 training, validation, and exploratory testing of five selected methods. The study examines class weighting, focal and asymmetric losses, threshold selection, label-aware attention, sentence–label contrastive alignment, dropout consistency, label graphs, adversarial perturbations, and layer-wise learning rates. All reported measurements are extracted from 51 completed cross-validation runs and 15 final-training runs, with aggregate scores checked against fold-level evidence. CodeBERT with R-Drop leads Python and Java screening, whereas RoBERTa with Focal Loss leads Pharo screening. Final-test micro F1 is highest for CodeBERT with R-Drop on Python (65.41 percent), CodeBERT with Focal Loss on Java (70.34 percent), and the RoBERTa BCE baseline on Pharo (73.71 percent). Per-label analysis shows that macro and micro improvements can disagree. Comparable classical-machine-learning results are unavailable and are explicitly marked TBD. Because full-population screening precedes the final split, the test population is not independent of method selection. Conclusions are therefore exploratory and motivate grouped confirmatory evaluation, repeated seeds, and validation-controlled ensembles.

Keywords: code comments; multi-label classification; RoBERTa; CodeBERT; cross-validation; class imbalance.

CONTENTS

Abstract i

List of Abbreviations viii

1 Introduction 1

1.1 Background and Motivation 1

1.1.1 Relevant Concepts 1

1.2 Problem Statement 1

1.3 Research Objectives 2

1.4 Research Questions and Contributions 2

1.5 Report Organization 2

2 Related Work 3

2.1 Code Comment Classification 3

2.2 Traditional Machine Learning Approaches 3

2.3 Transformer-based Approaches 4

2.4 RoBERTa and CodeBERT 4

2.5 Multi-label Learning Techniques 4

2.6 Research Gaps 5

3 Dataset and Problem Formulation 6

3.1 NLBSE'23 Dataset Description 6

3.2 Exploratory Data Analysis 7

3.3 Data Preprocessing 8

3.4 Label Distribution and Class Imbalance 9

3.5 Multi-label Problem Formulation 10

3.6 Data Splitting Strategy 11

4 Methodology 12

4.1 Overall Research Framework 12

4.2 Traditional Machine Learning Methods 12

4.2.1 TF-IDF Representation 12

4.2.2 Random Forest and Logistic Regression 12

4.2.3 Linear SVM, XGBoost, and Classifier Chains 13

4.2.4 Planned Objectives and Controlled Comparison 13
4.3 Transformer-based Deep Learning Methods 14
4.3.1 Backbones and Multi-label Head 14
4.3.2 Binary Cross-Entropy Objective 14
4.4 Improvement Techniques 15
4.4.1 Weighted BCE and Focal Loss 15
4.4.2 Threshold Tuning 15
4.4.3 Label-Aware Attention and Contrastive Learning 15
4.4.4 Consistency, Graphs, and Other Improvements 16
4.5 Model Architecture and Training Pipeline 17
4.6 Optimization and Hyperparameter Settings 18
5 Experimental Setup 19
5.1 Experimental Environment 19
5.2 Dataset Partitioning 19
5.3 Stage 1: Five-Fold Screening on All Data 19
5.4 Stage 2: Fixed Train/Validation/Test Training 20
5.5 Evaluation Metrics 20
5.5.1 Micro and Macro F1 20
5.5.2 Macro Average Precision and Hamming Loss 20
5.5.3 Cross-validation Mean and Standard Deviation 21
5.6 Model Selection and Early Stopping 21
5.7 Experimental Reproducibility 22
6 Results and Discussion 23
6.1 Machine Learning Baseline Results 23
6.2 Stage 1: Baseline Cross-validation Results 23
6.3 Stage 1: Comparison of Improvements 24
6.4 Stage 2: Fixed-Split Validation and Test Results 27
6.5 Ablation Studies 29
6.5.1 Loss and Attention Contrasts 29
6.5.2 Consistency and Optimization Contrasts 29
6.6 Per-label Performance Analysis 31
6.7 Error Analysis 31

6.8 Discussion of Research Questions 32
6.9 Limitations and Threats to Validity 33
7 Conclusion and Future Work 34
7.1 Summary of the Research 34
7.2 Main Findings 34
7.3 Research Limitations 34
7.4 Future Research Directions 34
References 36
A Hyperparameter Configurations 38
B Detailed Cross-validation Results 39
C Per-label Evaluation Results 45
D Additional Figures and Updated EDA Evidence 47
D.1 Execution Coverage and Numerical Validation 49
D.2 Positive Support and Text Diversity 50
D.3 Normalized Label Association 52
D.4 Class Concentration and Generalization Scope 55
D.5 Fixed Stage-2 Label Balance 56
D.6 Original Category-specific Partitions 58
D.7 Python Conditional Relations and Group Constraints 60
D.8 Reproducibility and Evidence Boundaries 62

LIST OF FIGURES

3.1 Population size and positive-target density; color identifies source language. 7

3.2 Full empirical token-length distributions; the dashed reference marks the configured 128-token limit. 7

3.3 Membership in repeated-text groups. Gray denotes singleton-text sentences; blue denotes all repeated-group members. 8

3.4 Cardinality distributions: bar heights are percentages and annotations are counts. 9

3.5 Largest-group positive support versus sample SD of validation prevalence across the actual five folds. SD is in percentage points and is not F1 variability. 10

3.6 Java validation-label shifts in the actual CV folds; blue is above and brown below the population prevalence. 11

4.1 Implemented two-stage framework. 12

6.1 Stage-1 CV micro F1 and fold SD. 24

6.2 Exploratory stage-2 test micro F1 changes versus E1. 27

6.3 Final-stage training loss and validation micro F1. 30

6.4 Test per-label F1 of the selected methods. 31

D.1 Python: positive sentence IDs and distinct positive text groups; annotations report both counts. 50

D.2 Java: positive sentence IDs and distinct positive text groups; annotations report both counts. 51

D.3 Pharo: positive sentence IDs and distinct positive text groups; annotations report both counts. 51

D.4 Python: Jaccard similarity with a shared scale and omitted diagonal. 52

D.5 Java: Jaccard similarity with a shared scale and omitted diagonal. 53

D.6 Pharo: Jaccard similarity with a shared scale and omitted diagonal. 54

D.7 Cumulative concentration of sentences across source classes. 55

D.8 Fraction of test class names also present in training, with count denominators. 55

D.9 Python: signed stage-2 prevalence shifts; legend identifies split and sample size. 56

D.10 Java: signed stage-2 prevalence shifts; legend identifies split and sample size. 57
D.11 Pharo: signed stage-2 prevalence shifts; legend identifies split and sample size. 57
D.12 Python: positive rates in original category-specific train/test partitions. 58
D.13 Java: positive rates in original category-specific train/test partitions. 58
D.14 Pharo: positive rates in original category-specific train/test partitions. 59
D.15 Python conditional co-occurrence; row label is the conditioning event. 60
D.16 Python positive-label support attributable to the largest exact-text group. 61

LIST OF TABLES

3.1 Audited sentence populations and fixed stage-2 partitions. 6
3.2 Exact normalized-text group structure. 8
3.3 Complete label inventory and positive support. 9
4.1 Implemented DL experiment registry. 17
6.1 ML baselines: comparable measurements unavailable (TBD). 23
6.2 Python: stage-1 screening results. 25
6.3 Java: stage-1 screening results. 25
6.4 Pharo: stage-1 screening results. 26
6.5 Stage-2 validation, test changes, and best epoch. 27
6.6 Python: stage-2 exploratory test results. 28
6.7 Java: stage-2 exploratory test results. 28
6.8 Pharo: stage-2 exploratory test results. 29
6.9 Observed test confusion counts. 32
A.1 Recorded hyperparameter configurations. 38
B.1 All 255 fold-level validation results. 39
C.1 Complete per-label test evaluation. 45
D.1 Python: stage-1 CV and available exploratory stage-2 test scores. F1/AP in percent; – denotes not evaluated in stage 2. 47
D.2 Java: stage-1 CV and available exploratory stage-2 test scores. F1/AP in percent; – denotes not evaluated in stage 2. 48
D.3 Pharo: stage-1 CV and available exploratory stage-2 test scores. F1/AP in percent; – denotes not evaluated in stage 2. 48
D.4 Executed revised notebooks; every analysis cell completed successfully. 49

LIST OF ABBREVIATIONS

AP	Average Precision
BCE	Binary Cross-Entropy
CV	Cross-Validation
DL	Deep Learning
EDA	Exploratory Data Analysis
GCN	Graph Convolutional Network
HL	Hamming Loss
LLRD	Layer-wise Learning-Rate Decay
ML	Machine Learning
NLP	Natural Language Processing
OOF	Out-of-Fold
SD	Standard Deviation
SVM	Support Vector Machine
TF-IDF	Term Frequency–Inverse Document Frequency
TBD	To Be Determined
TP/FP/FN/TN	True/False Positive/Negative
pp	Percentage points

CHAPTER 1. INTRODUCTION

1.1 Background and Motivation

Software comments communicate intent, usage constraints, design decisions, and implementation knowledge that are difficult to recover from executable statements alone. However, a class comment may combine several kinds of information in a short passage. Automatically identifying those information types can support documentation navigation, maintenance, and the organization of software knowledge. The practical target is therefore not to assign a single topic to an entire repository, but to identify the potentially overlapping roles of individual comment sentences.

The NLBSE'23 code-comment competition provides a concrete setting for this problem: 6,738 sentences from Python, Java, and Pharo, with language-specific information categories [1]. Programming language identifies the source ecosystem; it does not mean that each comment is written in a different human language. A sentence can carry several annotations, so a classifier must preserve multiple positive labels rather than force a mutually exclusive decision.

1.1.1 Relevant Concepts

Multi-label classification differs from multiclass classification in both output interpretation and evaluation. The output is a binary vector, and each coordinate represents an independent annotation decision conditional on the input. Label dependence can still be learned through shared representations or explicit graphs. In imbalanced data, a model may obtain a low per-target error while failing on rare labels; consequently, micro F1, macro F1, ranking quality, and Hamming loss must be interpreted together.

1.2 Problem Statement

Given the text of one comment sentence and a specified source-language subset, the system predicts the subset of applicable information categories. Class names and sentence identifiers are retained for auditing but are not supplied as model features. The implemented DL experiments use a separate model for each programming language, with five outputs for Python and seven for each of Java and Pharo. Encoder depth and the number of output labels are distinct quantities.

The central difficulty is to determine which improvements survive a common evaluation procedure rather than merely increasing architectural complexity. This

report audits existing code and measurements: 51 completed stage-1 runs and 15 completed stage-2 runs. It does not retrospectively manufacture missing ML scores, sentence-level predictions, statistical significance, or an untouched test set. These evidence boundaries are part of the problem definition.

1.3 Research Objectives

The study aims to establish an auditable multi-label data representation, examine conventional TF-IDF classifiers, and compare pretrained RoBERTa and CodeBERT with loss, attention, consistency, graph, and optimization modifications. Five-fold screening measures variability across partitions; fixed train/validation/test training then characterizes five selected methods. The report also investigates rare-label behavior and identifies implementation choices that can invalidate otherwise plausible comparisons.

1.4 Research Questions and Contributions

RQ1 asks whether code-oriented pretraining offers a consistent advantage over a general-language encoder. RQ2 asks whether imbalance-aware objectives, label dependence, or dropout consistency improve the corresponding baseline. RQ3 asks whether rankings remain stable across programming languages and the two evaluation stages. RQ4 concerns the relationship between aggregate scores, rare-label errors, and the limitations of the experimental protocol.

The contribution is a reproducible empirical analysis rather than a claim of a new foundation model. It consolidates sentence-level targets, enforces exact-text group separation within each experiment, documents seventeen DL variants, and reconciles every reported score with its source metrics. It also distinguishes conceptual ML baselines from evaluated systems and explicitly identifies method-selection contamination of the second-stage test population.

1.5 Report Organization

Chapter 2 reviews the methodological foundations. Chapter 3 audits the corpus and formalizes the task. Chapter 4 describes implemented models and planned baselines, while Chapter 5 specifies evaluation and reproducibility. Chapter 6 presents measured results, ablations, and limitations. Chapter 7 summarizes supported findings and future work. Appendices preserve detailed settings, all fold scores, per-label results, and evidence provenance.

CHAPTER 2. RELATED WORK

2.1 Code Comment Classification

The official competition frames code-comment classification as a set of binary decisions over information types [1]. Its repository documents category-specific data preparation and Random Forest baselines using text and NLP features [2]. Those systems are relevant historical comparators, but they are not interchangeable with a joint multi-label classifier evaluated on a newly constructed group split. A claim of outperforming the official baseline would require reproducing the same partitions, features, and aggregation rules.

Sentence classification also discards context that may be essential for interpretation. A phrase naming an API can be a usage instruction in one class and a cross-reference in another. The current text-only input therefore tests information recoverable from the sentence itself, not complete program understanding. This distinction prevents the study from attributing all errors to model capacity when some ambiguity may arise from missing context or annotation conventions.

2.2 Traditional Machine Learning Approaches

TF-IDF represents a document by lexical frequency adjusted for corpus prevalence [3]. With word and character n-grams, a sparse model can exploit documentation vocabulary and fragments of identifiers without learning a contextual encoder. Logistic regression and linear support-vector methods are natural candidates for high-dimensional sparse representations; scalable linear classification is established by LIBLINEAR [4]. Their role here is to quantify the value of lexical evidence under exactly the same split as the neural systems.

Random Forest aggregates randomized decision trees [5], whereas XGBoost adds regularized trees sequentially [6]. These methods can capture interactions between features, although sparse lexical spaces and small training sets require careful complexity control. The present repository does not contain a verified comparable run for either TF-IDF variant. Their discussion is therefore methodological and prospective, rather than an empirical assertion that a tree ensemble is stronger or weaker than a Transformer.

Binary relevance fits one classifier per label. Classifier chains instead expose earlier labels as additional inputs to later classifiers [7]. This creates a controlled clas-

sical counterpart to graph-based label interaction, but introduces ordering and error-propagation effects. A fair comparison must keep vocabulary learning inside each fold and evaluate several predefined chain orders or an order selected without access to the evaluation targets.

2.3 Transformer-based Approaches

Self-attention constructs context-dependent token representations through interactions among sequence positions [8]. A pretrained encoder can therefore assign different representations to the same lexical item in different sentences. Fine-tuning transfers those representations to the target annotation system. Nevertheless, short comments, repeated phrases, and limited labeled data mean that a larger hypothesis space does not automatically yield better generalization.

2.4 RoBERTa and CodeBERT

RoBERTa revisits masked-language-model pretraining and demonstrates the importance of training data and optimization choices [9]. CodeBERT incorporates programming-language and natural-language data with a hybrid pretraining objective [10]. These pretraining differences motivate a comparison, but do not establish that CodeBERT must dominate on comments. The input in this project contains no paired method body, and Pharo is not one of the six programming languages listed for CodeBERT pretraining.

The implementation uses the base checkpoints as feature initializations and fine-tunes their parameters end to end. Separate classification heads isolate the five- or seven-label tasks. Because the same output dimensionality can be attached to either backbone, changes in performance cannot be explained by counting encoder layers as classes. Tokenization, checkpoint initialization, loss reduction, and optimizer settings are more relevant implementation variables.

2.5 Multi-label Learning Techniques

Multi-label learning requires attention to label imbalance, dependence, and decision thresholds [11]. Focal Loss reduces the contribution of easy examples [12]; asymmetric objectives treat positive and negative labels differently [13]. Label-wise attention provides a precedent for selecting different text evidence for different outputs [14]. The present attention and sentence–label contrastive heads are project-specific adaptations, not reproductions of those original architectures.

R-Drop promotes agreement between stochastic dropout predictions [15].

Graph-based multi-label recognition offers a way to encode label co-occurrence [16], and embedding-level adversarial training motivates perturbation-based regularization [17]. Each technique changes a different aspect of learning. Combining them without controlled contrasts would obscure whether a gain arises from the representation, the objective, the optimizer, or additional computation.

2.6 Research Gaps

The immediate gap is not the absence of candidate architectures, but the absence of a complete and comparable evidence chain. Classical baselines have not yet been rerun on the shared grouped data; independent confirmation is unavailable after full-data screening; and the observed DL differences come from one base seed. A useful contribution must first close these evaluation gaps before claiming a general solution across programming ecosystems.

CHAPTER 3. DATASET AND PROBLEM FORMULATION

3.1 NLBSE'23 Dataset Description

The corpus contains 6,738 class-comment sentences from Python, Java, and Pharo. The official competition describes language-specific binary category tasks [1,2]. In this project, the category-wise records are pivoted into one complete multi-label vector per sentence ID. The category name identifies the binary task; only instance_type determines whether the corresponding label is positive. Class names are retained for auditing and are not model inputs.

The revised EDA reads only the canonical raw CSVs under `data/raw/nlbse2023`. It validates the raw schema, binary targets, one record per ID/category pair, complete label coverage, and consistent text/class metadata. It rejects invalid records instead of silently discarding them. All three revised notebooks completed successfully; their 27 analysis cells produced 35 figures. Table 3.1 describes the population and the fixed final split.

Table 3.1: Audited sentence populations and fixed stage-2 partitions.

Language	N	L	Train	Val.	Test	Cardinality
Python	2555	5	2044	255	256	1.121
Java	2418	7	1935	241	242	1.177
Pharo	1765	7	1413	176	176	1.158

Python has five outputs; Java and Pharo each have seven. The nineteen language-scoped label names do not form a nineteen-output task for every sentence and are unrelated to the number of Transformer layers. The label systems and sentence populations are preserved throughout the reported experiments. The revised visual analyses change neither annotation values nor partition membership.

3.2 Exploratory Data Analysis

Figure 3.1 separates population size from target density. Density divides the number of positive decisions by NL, while cardinality divides by N only. Neither statistic measures classification quality. Language colors remain fixed across the EDA: blue for Python, muted green for Java, and ochre for Pharo.

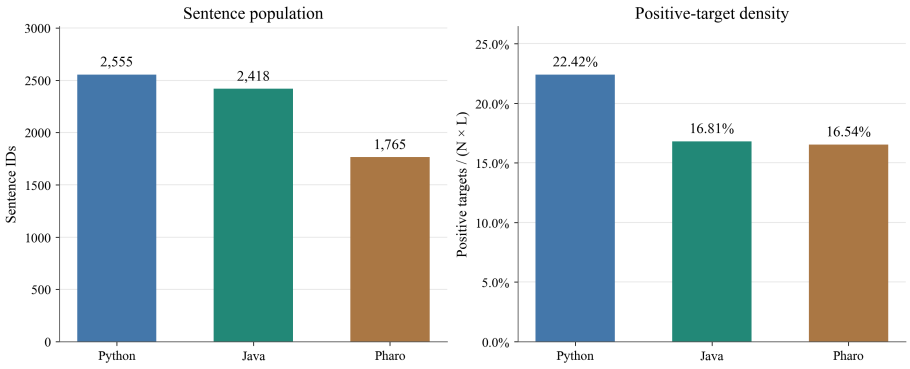


Figure 3.1: Population size and positive-target density; color identifies source language.

Figure 3.2 uses the saved E1 RoBERTa tokenizer locally and counts special tokens before truncation. Only five Java sentences exceed 128 tokens; none do in Python or Pharo. Word counts are analyzed separately and are not substituted for subword-token lengths.

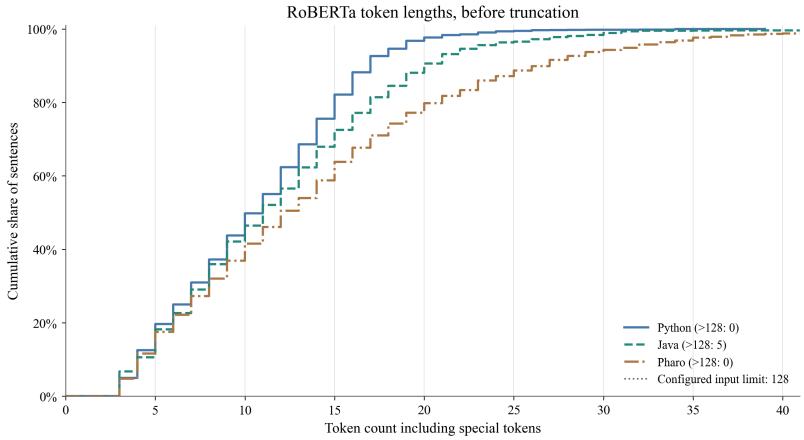


Figure 3.2: Full empirical token-length distributions; the dashed reference marks the configured 128-token limit.

3.3 Data Preprocessing

Text preprocessing collapses whitespace while retaining the case and symbols present in the supplied data. Equal normalized texts define indivisible groups. Sentence IDs remain distinct and targets are preserved, including disagreements between records with identical text. Such disagreement can reflect context-dependent annotation; it is not automatically an annotation error.

Table 3.2 distinguishes repeated groups, all their members, and excess copies beyond the first record. These quantities answer different questions. Figure 3.3 reports member fractions, not the fraction of duplicate rows in the long-format file. Schema checks find no missing cells, exact duplicate long rows, empty texts, or inconsistent ID metadata.

Table 3.2: Exact normalized-text group structure.

Language	Groups	Repeated	Members	Excess	Largest
Python	2299	112	368	256	19
Java	1668	252	1002	750	51
Pharo	1620	60	205	145	18

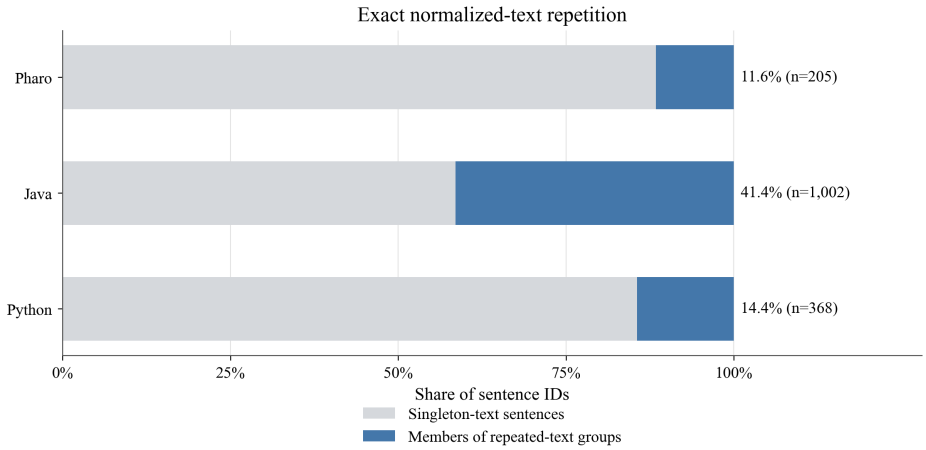


Figure 3.3: Membership in repeated-text groups. Gray denotes singleton-text sentences; blue denotes all repeated-group members.

3.4 Label Distribution and Class Imbalance

Table 3.3 preserves the exact ordered label spellings used by the models. Imbalance is described through positive support and prevalence, not a subjective difficulty score. Macro and micro evaluation therefore remain complementary. Figure 3.4 confirms that multiple labels occur even though single-positive sentences are most frequent.

Table 3.3: Complete label inventory and positive support.

Language	ID	Label	n	%
Python	1	DevelopmentNotes	312	12.21
Python	2	Expand	504	19.73
Python	3	Parameters	794	31.08
Python	4	Summary	454	17.77
Python	5	Usage	800	31.31
Java	1	Expand	632	26.14
Java	2	Ownership	115	4.76
Java	3	Pointer	364	15.05
Java	4	deprecation	127	5.25
Java	5	rational	280	11.58
Java	6	summary	415	17.16
Java	7	usage	912	37.72
Pharo	1	Classreferences	77	4.36
Pharo	2	Collaborators	127	7.20
Pharo	3	Example	748	42.38
Pharo	4	Intent	218	12.35
Pharo	5	Keyimplementationpoints	232	13.14
Pharo	6	Keymessages	305	17.28
Pharo	7	Responsibilities	336	19.04

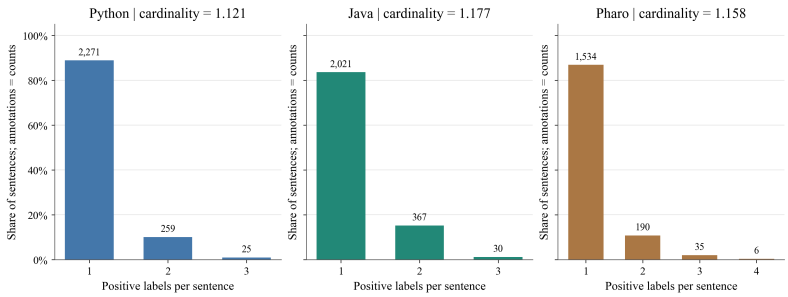


Figure 3.4: Cardinality distributions: bar heights are percentages and annotations are counts.

3.5 Multi-label Problem Formulation

$$\mathcal{D}_\ell = \{(x_i, \mathbf{y}_i)\}_{i=1}^{N_\ell}, \qquad \mathbf{y}_i \in \{0,1\}^{L_\ell}. \tag{3.1}$$

Here ℓ indexes the source-language subset, x_i is normalized sentence text, N_ℓ is its population size, and L_ℓ is its label count. Scores $p_{ij} \in [0,1]$ are converted to binary decisions by $\hat{y}_{ij} = \mathbb{1}[p_{ij} \geq t_j]$, where t_j is a fixed or validation-selected threshold. Independent sigmoid outputs permit simultaneous positives; they do not assert statistical independence of the observed labels.

Label association is examined with Jaccard similarity, $J_{jk} = n_{jk}/(n_j + n_k - n_{jk})$, where n_j is positive support and n_{jk} is joint support. All count products use 64-bit integers, and their diagonals are checked against label support. The plots omit trivial diagonal entries. Association and conditional co-occurrence are descriptive, not causal relations or model predictions.

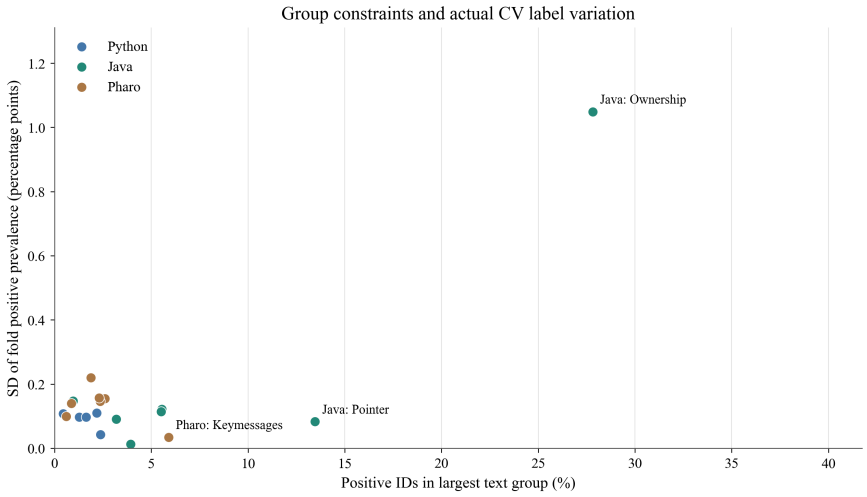


Figure 3.5: Largest-group positive support versus sample SD of validation prevalence across the actual five folds. SD is in percentage points and is not F1 variability.

Figure 3.5 identifies the practical constraint introduced by grouping. The Java Ownership label has 32 of 115 positives in one text group. A group-preserving split cannot distribute these records independently. This observation supports auditing fold balance but does not establish that a label is intrinsically hard to classify.

3.6 Data Splitting Strategy

Stage 1 uses five prepared grouped folds over the entire language population. Every sentence appears in validation once, and identical normalized texts remain on one side of each fold. The revised notebooks read the actual folds.json; they do not construct an illustrative random KFold substitute. The largest observed deviation is associated with grouped Ownership annotations in Java (Figure 3.6).

Stage 2 separately partitions the same population into approximately 80% training, 10% validation, and 10% test. Raw targets, normalized text, unique IDs, full population coverage, and within-stage text-group separation are checked before visualization. Signed prevalence differences are reported in percentage points relative to the complete population; they are not performance differences.

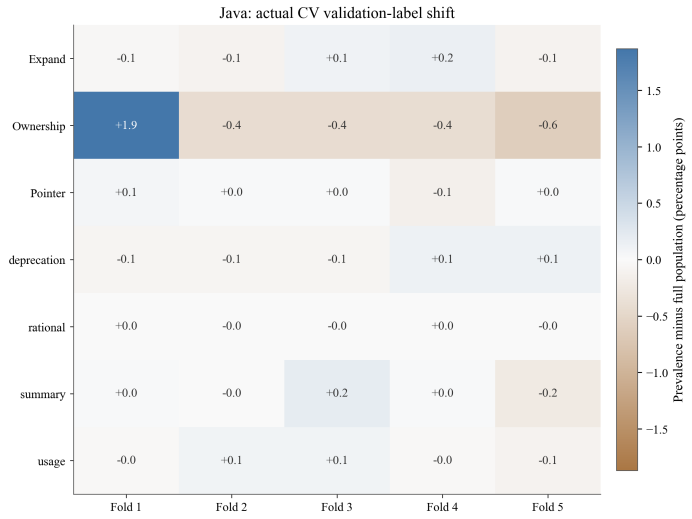


Figure 3.6: Java validation-label shifts in the actual CV folds; blue is above and brown below the population prevalence.

The original raw partitions remain distinct category-specific tasks and are analyzed separately in Appendix D. Class names overlap between final training and test, so exact-text separation is not project-level independence. Furthermore, final test rows already participated in stage-1 method selection. Accordingly, the stage-2 test measurements are exploratory; confirmatory evaluation requires a holdout defined before all method development.

CHAPTER 4. METHODOLOGY

4.1 Overall Research Framework

The framework separates data validation, model screening, final fitting, and result auditing. Shared language-specific targets and folds make comparisons interpretable within stage 1. The second stage then restarts selected systems from their pretrained initialization and fits them only on the fixed training partition. Figure 4.1 summarizes the implemented process and explicitly marks the cross-stage reuse that limits independent inference.

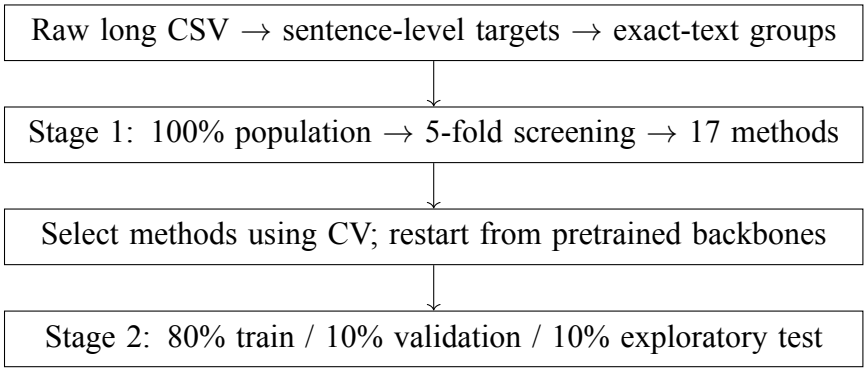


Figure 4.1: Implemented framework. Later test rows were included in stage-1 screening.

4.2 Traditional Machine Learning Methods

4.2.1 TF-IDF Representation

$$v_{it} = \mathrm{tf}(t, x_i)\left[\log\frac{1+n}{1+\mathrm{df}(t)}+1\right], \qquad \tilde{\mathbf{v}}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|_2}. \tag{4.1}$$

Here n is the fold-training size, $\mathrm{df}(t)$ counts training documents containing term t, and tf is term frequency. The maintained classical pipeline combines word unigrams/bigrams and character-within-word 3–5-grams, with a minimum document frequency of two and at most 20,000 features in each branch. The vectorizer is refitted per fold. These settings are implementation evidence, not completed results on the new grouped protocol.

4.2.2 Random Forest and Logistic Regression

A TF-IDF Random Forest baseline would average tree probabilities for each binary label and tune tree count, depth, and feature subsampling within training data. It is planned rather than measured here, and it should not be confused with the official

text-plus-NLP Weka system. Logistic regression is implemented as one-versus-rest, with balanced class weights, $C = 5$, and a maximum of 2,000 iterations. Its linear decision boundary provides an interpretable lexical reference.

$$p_j(\mathbf{v}) = \sigma(\mathbf{w}_j^\top \mathbf{v} + b_j), \qquad \mathcal{L}_j = \sum_i \mathrm{BCE}(y_{ij}, p_j(\mathbf{v}_i)) + \lambda \|\mathbf{w}_j\|_2^2. \tag{4.2}$$

In Equation (4.2), $\mathbf{w}_j$ and b_j are label-specific coefficients and intercepts, while λ controls regularization. Balanced weighting compensates for target frequency but need not improve thresholded F1. Probabilities and ranking metrics must be inspected separately from binary accuracy. A calibration or threshold experiment would require validation predictions, rather than tuning on the reported test labels.

4.2.3 Linear SVM, XGBoost, and Classifier Chains

The implemented linear SVM uses balanced weights and $C = 1$. Its decision function can support ranking metrics without being interpreted as a calibrated probability. XGBoost is a planned one-versus-rest extension with regularized boosted trees; no training configuration or score is reported as executed. Classifier chains use a logistic base estimator and a randomized order fixed by seed 42. The final classifier must consume earlier predicted labels at inference, which can propagate upstream errors.

$$P(\mathbf{y} \mid \mathbf{v}) = \prod_{k=1}^{L} P\left(y_{\pi(k)} \mid \mathbf{v}, y_{\pi(1)}, \ldots, y_{\pi(k-1)}\right). \tag{4.3}$$

The permutation π in Equation (4.3) defines a chain ordering. The additional implemented SGD classifier uses logistic loss and balanced class weighting. Legacy scripts with category-string parsing or vocabulary fitting before CV are excluded from evidence: they can change the labels or expose evaluation text to preprocessing. Integrating all classical models into the shared group loader remains required before making an ML-versus-DL performance claim.

4.2.4 Planned Objectives and Controlled Comparison

$$\hat{p}_j(\mathbf{v}) = \frac{1}{B} \sum_{b=1}^{B} T_{bj}(\mathbf{v}), \qquad f_j^{(M)}(\mathbf{v}) = \sum_{m=1}^{M} \nu h_{mj}(\mathbf{v}). \tag{4.4}$$

The first expression averages the label probabilities of B randomized trees T_{bj} in Random Forest. The second denotes an additive boosted-tree score, with M trees h_{mj} and shrinkage ν; a logistic link converts the score to a probability for a binary

target. Tree counts and regularization strengths remain unspecified because these baselines have not been trained in the audited protocol. They must be selected on training-side validation rather than inferred from Transformer settings.

$$\min_{\mathbf{w}_j, b_j} \frac{1}{2}\|\mathbf{w}_j\|_2^2 + C \sum_i \max\{0, 1 - \tilde{y}_{ij}(\mathbf{w}_j^\top \mathbf{v}_i + b_j)\}^2. \tag{4.5}$$

For the squared-hinge linear SVM, $\tilde{y}_{ij} = 2y_{ij} - 1$ is the signed target and C trades margin regularization against training violations. Balanced weighting rescales the example terms in the actual classifier. This margin objective is distinct from probability estimation, so a calibration experiment should be separated from the underlying classifier comparison.

4.3 Transformer-based Deep Learning Methods

4.3.1 Backbones and Multi-label Head

$$\mathbf{H}_i = f_\theta(x_i) \in \mathbb{R}^{T_i \times d}, \quad \mathbf{z}_i = \mathbf{W}\mathbf{h}_{i,0} + \mathbf{b}, \quad p_{ij} = \sigma(z_{ij}), \tag{4.6}$$

The saved configurations use $d = 768$, a maximum length of 128, and RoBERTa-base or microsoft/codebert-base. T_i denotes the padded token length, $\mathbf{h}_{i,0}$ the first-token representation, and $\mathbf{W} \in \mathbb{R}^{L \times 768}$ the task head. Independent sigmoids permit simultaneous positive labels. A softmax would force competing labels to sum to one and would therefore impose the wrong output constraint.

4.3.2 Binary Cross-Entropy Objective

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{nL} \sum_{i=1}^{n} \sum_{j=1}^{L} \left[y_{ij} \log p_{ij} + (1 - y_{ij}) \log(1 - p_{ij}) \right]. \tag{4.7}$$

Training evaluates BCE directly from logits for numerical stability. The library supports masks for unobserved targets, but the prepared language-specific datasets have complete binary label vectors. E1 and E11 share this head and objective, differing in pretrained backbone. Each fold initializes a fresh model; the final training stage also restarts from pretrained parameters rather than continuing a fold checkpoint.

4.4 Improvement Techniques

4.4.1 Weighted BCE and Focal Loss

$$\mathcal{L}_{\text{WBCE}} = -\frac{1}{nL}\sum_{i,j}\left[a_j y_{ij}\log p_{ij} + (1-y_{ij})\log(1-p_{ij})\right], \quad a_j = \frac{n_j^-}{n_j^+}. \tag{4.8}$$

The positive weight a_j uses only the active training subset. Labels with no positives or no negatives receive weight one to avoid non-finite loss. E2 changes the positive term of BCE, not the entire label loss. The expected effect is a different precision–recall balance for minority targets; whether it improves micro or macro F1 is an empirical question rather than a consequence of the weighting formula.

$$\mathcal{L}_{\text{focal}} = -\frac{1}{nL}\sum_{i,j}\alpha_{t,ij}(1-p_{t,ij})^{\gamma}\log p_{t,ij}, \quad p_{t,ij} = y_{ij}p_{ij} + (1-y_{ij})(1-p_{ij}). \tag{4.9}$$

E3 and E17 adapt Focal Loss to independent binary outputs, with $\gamma = 2$ and no alpha weighting, so $\alpha_{t,ij} = 1$. The mechanism emphasizes uncertain decisions, but uncertain observations may include annotation noise. The RoBERTa and CodeBERT versions permit a backbone comparison under the same loss. They do not demonstrate a benefit until compared with E1 and E11, respectively, on the shared protocol.

4.4.2 Threshold Tuning

$$t_j^{\star} = \arg\max_{t\in\mathcal{G}} F_{1,j}(\mathbb{1}[p_{ij} \geq t], y_{ij})\,. \tag{4.10}$$

The grid $\mathcal{G}$ ranges from 0.05 to 0.95 in steps of 0.05, with ties favoring proximity to 0.5. The current staged implementation deliberately evaluates E4 at 0.5 throughout stage 1 and defers threshold tuning to stage-2 validation. Because E4 was not among the five final methods, no threshold-tuned final result is available. Identical E1/E4 screening scores therefore reflect the implemented protocol, not evidence that threshold tuning is intrinsically ineffective.

4.4.3 Label-Aware Attention and Contrastive Learning

$$e_{ijt} = \frac{\mathbf{q}_j^{\top}\tanh(\mathbf{A}\mathbf{h}_{it})}{\sqrt{d}}, \quad \alpha_{ijt} = \text{softmax}_t(e_{ijt}), \quad \mathbf{c}_{ij} = \sum_t \alpha_{ijt}\mathbf{h}_{it}. \tag{4.11}$$

E5 learns a query $\mathbf{q}_j$ for each label, masks padding positions, and classifies the label-specific context $\mathbf{c}_{ij}$. E6 adds a normalized projected sentence representation and a similarity distribution over label queries. Its target distribution is uniform across the sentence's positive labels. The auxiliary term has weight 0.1 and temperature 0.1. This is sentence–label alignment, motivated by contrastive representation learning [18], not the pairwise supervised-contrastive objective applied verbatim.

$$\mathcal{L}_{\text{E6}} = \mathcal{L}_{\text{BCE}} - \lambda_c \frac{1}{|I_+|} \sum_{i \in I_+} \sum_{j=1}^{L} \frac{y_{ij}}{\sum_k y_{ik}} \log \frac{\exp(s_{ij}/\tau)}{\sum_k \exp(s_{ik}/\tau)}. \tag{4.12}$$

Here I_+ contains sentences with positive annotations, s_{ij} is cosine similarity between projected sentence and label query, τ is temperature, and λ_c scales the auxiliary term. The BCE branch still permits multiple positive labels; the auxiliary softmax operates only over the alignment task. This distinction avoids incorrectly treating the final classification output as mutually exclusive.

4.4.4 Consistency, Graphs, and Other Improvements

$$\mathcal{L}_{\text{RDrop}} = \tfrac{1}{2}(\mathcal{L}_{\text{BCE}}^{(1)} + \mathcal{L}_{\text{BCE}}^{(2)}) + \tfrac{\lambda}{2}\left[D_{\text{KL}}(P\|Q) + D_{\text{KL}}(Q\|P)\right]. \tag{4.13}$$

E8 and E15 perform two dropout-enabled training passes and use symmetric KL divergence between independent Bernoulli predictions, with $\lambda = 1$. Inference uses one pass. E7 uses asymmetric focusing $(\gamma_+, \gamma_-) = (1, 4)$ and negative clipping 0.05, with detached focusing weights. E10 adds a supervised embedding perturbation normalized per sentence with padding masked out; it is not a full virtual-adversarial or projected-gradient procedure.

$$\ell_{\text{ASL}} = -y(1-p)^{\gamma_+} \log p - (1-y)(1-q)^{\gamma_-} \log q, \qquad q = \min(1, 1-p+c). \tag{4.14}$$

In the asymmetric loss, c clips negative probabilities and γ_+/γ_- control positive/negative focusing. For E10, let $\mathbf{g}_i$ be the gradient of supervised loss with respect to non-padding token embeddings. The perturbation is $\boldsymbol{\delta}_i = \epsilon \mathbf{g}_i / (\|\mathbf{g}_i\|_2 + \varepsilon)$, where $\epsilon = 1$ is the configured perturbation magnitude and ε protects normalization. Clean and perturbed supervised losses are accumulated before the update; this adds training cost and cannot be treated as an equal-compute change.

$$C_{jk} = \frac{(\mathbf{Y}^\top \mathbf{Y})_{jk}}{\sqrt{s_j s_k}}, \quad \widetilde{\mathbf{A}} = \mathbf{D}^{-1/2}(\text{threshold}(\mathbf{C}) + \mathbf{I})\mathbf{D}^{-1/2}. \tag{4.15}$$

For graph construction, $\mathbf{Y}$ contains only current-training labels and $s_j = \sum_i Y_{ij}$. Off-diagonal cosine co-occurrences below 0.1 are removed; self-loops and symmetric degree normalization follow, where $\mathbf{D}$ is the diagonal matrix of row sums after adding self-loops. E9 applies a two-layer residual GCN to learn classifier weights, initializing the correction branch to zero so logits start at baseline scale. E12 refines attention contexts with the same graph idea. These are adaptations of label-relation modeling, not exact ML-GCN replications.

4.5 Model Architecture and Training Pipeline

Table 4.1 lists all seventeen variants. The pipeline tokenizes each sentence, constructs a shuffled training loader, builds fold-specific weights or graph statistics, and runs a shared optimizer loop. Validation uses deterministic evaluation mode. The best parameter state is restored before metrics or final test evaluation. Model outputs, loss masks, label order, and data fingerprints are checked consistently across the language-specific runs.

Table 4.1: Implemented DL experiment registry.

ID	Architecture / intervention
E1	RoBERTa / BCE
E2	RoBERTa / Weighted BCE
E3	RoBERTa / Focal
E4	RoBERTa / Thresholds
E5	RoBERTa / Attention
E6	RoBERTa / Attention + Contrastive
E7	RoBERTa / ASL
E8	RoBERTa / R-Drop
E9	RoBERTa / Label GNN
E10	RoBERTa / FGM
E11	CodeBERT / BCE
E12	RoBERTa / Attention + GNN
E13	CodeBERT / Head LR
E14	CodeBERT / LLRD
E15	CodeBERT / R-Drop
E16	CodeBERT / LLRD + R-Drop
E17	CodeBERT / Focal

4.6 Optimization and Hyperparameter Settings

AdamW [19] uses a base learning rate of 2×10^{-5}, weight decay 0.01, linear scheduling, and a 10% warmup fraction. E13 raises head learning rate to 10^{-4}. E14 additionally decays rates toward earlier encoder layers by 0.9 per layer; E16 combines that schedule with R-Drop. E15 keeps the ordinary optimizer rather than inheriting E14. The implementation applies weight decay to all parameter groups, including biases and LayerNorm parameters, which is an explicit local choice.

$$\eta_k = \eta_{\text{enc}}\, \rho^{K-1-k}, \quad k = 0, \ldots, K-1; \qquad \eta_{\text{emb}} = \eta_{\text{enc}} \rho^{K}. \tag{4.16}$$

In Equation (4.16), K is encoder depth and $\rho = 0.9$ for layer-wise decay. The classification head has its separately configured rate. Batch sizes are 16 for training and 32 for evaluation, with gradient clipping at one and a maximum of ten epochs. These fixed settings bound the conclusions: a poor score does not show that a method cannot work under other hyperparameters.

CHAPTER 5. EXPERIMENTAL SETUP

5.1 Experimental Environment

The measured runs record device identifiers `cuda:0` and `cuda:1`, base seed 42, and their effective YAML configurations. The first GPU processes E1–E9 and the second E10–E17 during screening; the final stage uses E1/E3/E9 and E15/E17, respectively. Each process trains one model on one GPU; there is no distributed data parallelism. GPU model, historical driver version, and exact training package versions are not preserved in these metrics and are therefore not asserted.

Repository requirements specify PyTorch, Transformers, NumPy, PyYAML, TensorBoard, Matplotlib, pandas, and tqdm. These version ranges are installation constraints, not a historical environment lock. The evidence collector verifies existing measurements without training new models. An audit performed later may run under different software or CPU-only PyTorch, so it cannot recover the original CUDA environment from the currently installed packages.

5.2 Dataset Partitioning

All DL methods consume the same prepared data for a given language and stage. Source hashes, population fingerprints, concrete file fingerprints, and ordered labels are checked before aggregation. The E1/E3/E9/E15/E17 final subset was selected to retain a baseline and several contrasting approaches across languages, not to implement the exact per-language top-four selection manifest. This rationale must be distinguished from an automatic rule that would choose different candidate sets per language.

5.3 Stage 1: Five-Fold Screening on All Data

A screening run consists of five independently initialized training procedures on the fixed grouped folds. Approximately four fifths of the full language population train each model; the remaining fifth supplies epoch-level validation. Fold number k uses base seed $42 + k$, with zero-based indexing. Every sentence is held out once, but the training sets overlap. Consequently, fold scores are correlated and their standard deviation describes sensitivity to these partitions rather than uncertainty from independent repeated experiments.

The checkpoint yielding the largest validation micro F1 supplies the fold

score. This reuses validation for epoch selection and scoring, so the mean is a screening statistic rather than a nested estimate of the full selection procedure. No full-population model is fitted after stage-1 CV, and no test metric is produced in that stage. All 255 observed fold results are preserved in Appendix B.

5.4 Stage 2: Fixed Train/Validation/Test Training

Default early-stopping patience is two epochs. E9 instead uses patience three, requires at least five epochs before stopping, and uses micro AP to break exact F1 ties. The learning-rate schedule is planned for ten epochs even when early stopping terminates training sooner. These differences are recorded because an E9-versus-E1 contrast changes both the graph head and stopping policy; it is not an isolated causal test of graph structure.

The final stage trains only on its 80% training partition, selects the best state on 10% validation, and evaluates the remaining 10% after restoring that state. It neither merges validation into training nor uses the median CV best epoch. The recorded thresholds are 0.5 for all five final methods. Appendix A supplies the shared hyperparameter settings and method-specific deviations.

5.5 Evaluation Metrics

5.5.1 Micro and Macro F1

$$
\begin{aligned}
F_1^{\text{micro}} &= \frac{2\sum_j TP_j}{2\sum_j TP_j + \sum_j FP_j + \sum_j FN_j}, \\
F_1^{\text{macro}} &= \frac{1}{L}\sum_{j=1}^{L} \frac{2TP_j}{2TP_j + FP_j + FN_j}.
\end{aligned}
\tag{5.1}
$$

TP_j, FP_j, and FN_j count true positives, false positives, and false negatives for label j. Precision is $TP_j/(TP_j + FP_j)$ and recall is $TP_j/(TP_j + FN_j)$. The implementation returns zero for undefined F1 denominators. Micro F1 pools target decisions, while macro F1 gives rare labels the same weight as common labels. Neither should be substituted for the other when interpreting imbalance-related interventions.

5.5.2 Macro Average Precision and Hamming Loss

$$
AP_j = \sum_r \big(R_j(r) - R_j(r-1)\big) P_j(r), \qquad \text{mAP} = \frac{1}{|J_+|}\sum_{j \in J_+} AP_j. \tag{5.2}
$$

$P_j(r)$ and $R_j(r)$ are precision and recall at successive distinct score thresholds, and J_+ is the set of labels with at least one positive in the evaluated split. Tied scores are grouped at one threshold. AP is undefined for labels without positives, and the implementation excludes such labels from the macro mean while retaining an explicit null value. AP measures ranking quality, so improvements need not coincide with F1 at threshold 0.5.

$$\mathrm{HL} = \frac{1}{NL} \sum_{i=1}^{N} \sum_{j=1}^{L} \mathbb{1}[\hat{y}_{ij} \neq y_{ij}]. \tag{5.3}$$

Hamming loss counts the fraction of incorrect binary decisions. It rewards true negatives as well as correct positives and must therefore be read alongside target density. For example, an all-negative Pharo predictor would have population Hamming loss approximately 0.1654, yet would miss every positive annotation. Reported losses are proportions; F1 and AP tables use percentages unless stated otherwise.

5.5.3 Cross-validation Mean and Standard Deviation

$$\overline{m} = \frac{1}{5} \sum_{k=1}^{5} m_k, \qquad s = \sqrt{\frac{1}{4} \sum_{k=1}^{5} (m_k - \overline{m})^2}. \tag{5.4}$$

The report uses the sample standard deviation in Equation (5.4), matching the current DL summarizer. This differs from legacy ML code that computes a population standard deviation. The five fold scores are not used to claim statistical significance, and a single final run has no reported SD. A future repeated-seed design should distinguish fold variability from between-seed variability.

5.6 Model Selection and Early Stopping

Weights, learned label graphs, and any future TF-IDF vocabulary must be estimated from the training partition alone. Validation controls checkpoint choice and, where applicable, threshold choice. Test is not consulted by the training loop. Nevertheless, these local safeguards do not repair cross-stage dependence: screening methods on all rows before designating a test subset already exposes the final evaluation population to development decisions. This limitation is consistent with the general selection-bias concern described by Varma and Simon [20].

5.7 Experimental Reproducibility

The evidence audit checks completion flags, unique experiment keys (stage, language, model, and seed), equal within-language fingerprints, label order, group separation, and recomputed CV summaries. It preserves SHA-256 hashes of source metrics and split files. A reproducible rerun should retain effective configs, tokenizer artifacts, checkpoints, thresholds, library locks, hardware metadata, and per-sentence probabilities. Only the first group of these items is presently sufficient for numerical reconciliation; full historical environment reproduction remains incomplete.

CHAPTER 6. RESULTS AND DISCUSSION

6.1 Machine Learning Baseline Results

No comparable ML metrics remain for the current grouped five-fold and fixed 80/10/10 protocol. Table 6.1 therefore uses TBD rather than importing deleted historical results or official baseline figures from another evaluation design. This absence prevents a numerical conclusion that Transformers outperform TF-IDF classifiers in this project. The implemented classical pipeline is a starting point for future experiments, not an evaluated competitor in the present leaderboard.

Table 6.1: ML baselines: comparable measurements are unavailable (TBD).

Method	CV micro F1 ± SD	CV macro F1 ± SD	Test micro F1	Test macro F1	Test macro AP	Test HL
Random Forest	TBD	TBD	TBD	TBD	TBD	TBD
Logistic Regression	TBD	TBD	TBD	TBD	TBD	TBD
Linear SVM	TBD	TBD	TBD	TBD	TBD	TBD
XGBoost	TBD	TBD	TBD	TBD	TBD	TBD
Classifier Chains	TBD	TBD	TBD	TBD	TBD	TBD
SGD (logistic)	TBD	TBD	TBD	TBD	TBD	TBD

Completing this comparison requires all six baseline families to use the shared sentence-level targets and exact-text groups, learn vectorizers only within training folds, and expose scores suitable for AP. Linear SVM scores can be ranked directly; probability calibration is a separate validation-only operation. The same metric definitions and sample SD must be used across families before differences are interpreted as modeling effects.

6.2 Stage 1: Baseline Cross-validation Results

The BCE baseline comparison is language dependent. E11 exceeds E1 in Python CV micro F1, 63.97% versus 61.69%, but Java differs only slightly, 59.80% versus 59.68%. In Pharo, E1 scores 70.44% compared with E11's 68.05%. These are observations under one seed and a fixed configuration; they do not establish a universal advantage of either pretraining corpus.

Tables 6.2–6.4 contain only stage-1 CV measurements. Every method has five validation-fold scores; no test is evaluated at this stage. Combined cross-stage tables

are retained separately in Appendix D.

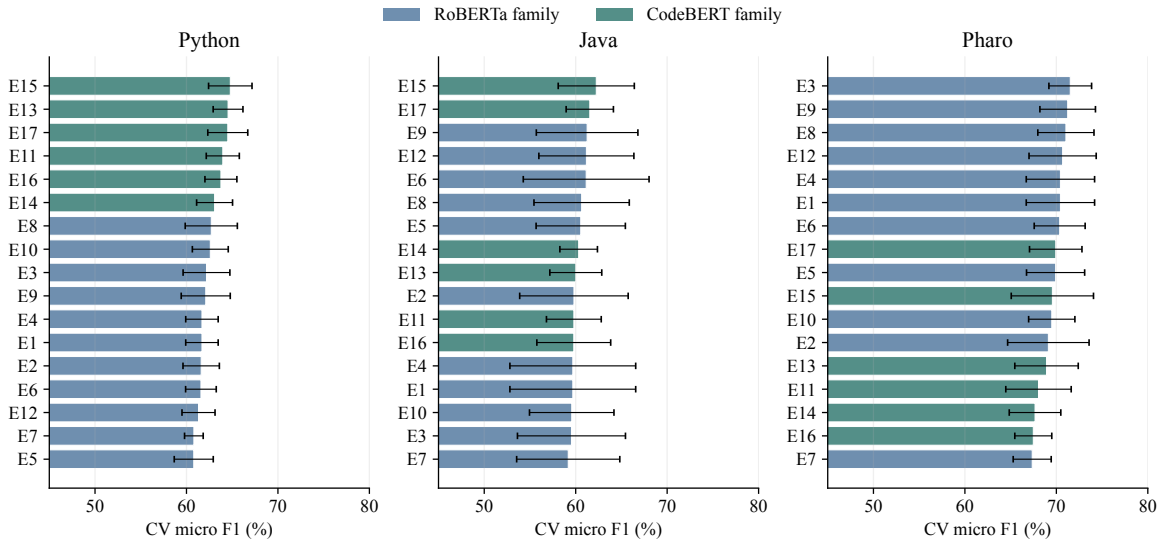


Figure 6.1: Stage-1 CV micro F1 with fold SD. Blue: RoBERTa family; teal: CodeBERT family. Error bars are not confidence intervals.

6.3 Stage 1: Comparison of Improvements

Stage-1 winners are E15 for Python (64.79% $\pm$ 2.37), E15 for Java (62.25% $\pm$ 4.16), and E3 for Pharo (71.52% $\pm$ 2.33). Figure 6.1 shows the entire ranking and fold variability. The four leading Python methods are all CodeBERT variants, while Pharo's leading four are RoBERTa variants. This contrast is consistent with language-specific behavior but does not identify its cause.

Table 6.2: Python: stage-1 screening only. F1/AP are percentages; SD is across five folds.

ID	CV micro F1 mean ± SD	CV macro F1 mean ± SD	CV macro AP mean ± SD	CV HL
E15	64.79 ± 2.37	58.61 ± 2.27	67.03 ± 1.94	0.1456
E13	64.55 ± 1.63	58.93 ± 1.61	66.39 ± 1.75	0.1499
E17	64.52 ± 2.19	59.38 ± 2.76	67.35 ± 2.17	0.1489
E11	63.97 ± 1.81	57.40 ± 3.31	66.04 ± 1.42	0.1494
E16	63.76 ± 1.75	55.81 ± 2.25	66.08 ± 1.90	0.1453
E14	63.07 ± 1.96	54.80 ± 3.09	64.49 ± 2.18	0.1504
E8	62.71 ± 2.85	56.83 ± 3.12	64.83 ± 2.63	0.1555
E10	62.60 ± 1.96	53.63 ± 1.35	64.28 ± 0.95	0.1504
E3	62.19 ± 2.56	56.53 ± 2.91	64.79 ± 1.98	0.1595
E9	62.11 ± 2.68	57.23 ± 2.19	63.63 ± 1.31	0.1633
E1	61.69 ± 1.77	56.12 ± 1.74	63.25 ± 2.01	0.1645
E4	61.69 ± 1.77	56.12 ± 1.74	63.25 ± 2.01	0.1645
E2	61.61 ± 1.99	58.34 ± 1.67	63.04 ± 0.53	0.1822
E6	61.58 ± 1.67	55.63 ± 2.45	63.34 ± 1.35	0.1619
E12	61.32 ± 1.81	55.97 ± 3.06	63.98 ± 0.65	0.1638
E7	60.81 ± 1.02	58.02 ± 0.82	65.63 ± 1.66	0.2182
E5	60.80 ± 2.13	54.51 ± 3.58	63.24 ± 1.51	0.1652

Table 6.3: Java: stage-1 screening only. F1/AP are percentages; SD is across five folds.

ID	CV micro F1 mean ± SD	CV macro F1 mean ± SD	CV macro AP mean ± SD	CV HL
E15	62.25 ± 4.16	64.95 ± 5.39	69.07 ± 4.90	0.1170
E17	61.54 ± 2.59	65.06 ± 4.24	70.00 ± 2.97	0.1206
E9	61.25 ± 5.55	64.69 ± 7.34	67.28 ± 6.26	0.1243
E12	61.17 ± 5.19	64.97 ± 4.98	67.70 ± 5.29	0.1273
E6	61.15 ± 6.88	65.57 ± 7.53	68.03 ± 6.42	0.1233
E8	60.65 ± 5.21	63.61 ± 6.22	68.41 ± 5.77	0.1234
E5	60.55 ± 4.88	64.06 ± 4.10	68.64 ± 5.44	0.1232
E14	60.32 ± 2.05	64.39 ± 3.26	69.77 ± 2.87	0.1219
E13	60.01 ± 2.84	64.22 ± 4.11	68.72 ± 3.16	0.1231
E2	59.81 ± 5.93	64.93 ± 5.89	66.17 ± 7.72	0.1583
E11	59.80 ± 3.00	62.62 ± 4.88	69.99 ± 3.26	0.1222
E16	59.79 ± 4.04	62.86 ± 5.06	69.79 ± 4.51	0.1190
E1	59.68 ± 6.88	62.73 ± 7.53	67.37 ± 6.29	0.1297
E4	59.68 ± 6.88	62.73 ± 7.53	67.37 ± 6.29	0.1297
E10	59.57 ± 4.62	63.24 ± 6.49	68.21 ± 5.17	0.1264
E3	59.54 ± 5.90	63.12 ± 6.86	67.24 ± 7.00	0.1277
E7	59.18 ± 5.64	63.37 ± 5.17	67.75 ± 6.88	0.1739

Table 6.4: Pharo: stage-1 screening only. F1/AP are percentages; SD is across five folds.

ID	CV micro F1 mean ± SD	CV macro F1 mean ± SD	CV macro AP mean ± SD	CV HL
E3	71.52 ± 2.33	62.67 ± 3.72	68.04 ± 4.52	0.0906
E9	71.24 ± 3.04	60.94 ± 4.64	66.59 ± 5.09	0.0909
E8	71.04 ± 3.07	60.71 ± 4.07	67.87 ± 4.28	0.0900
E12	70.68 ± 3.67	62.45 ± 5.93	66.46 ± 4.12	0.0938
E1	70.44 ± 3.74	60.53 ± 5.17	66.03 ± 5.62	0.0938
E4	70.44 ± 3.74	60.53 ± 5.17	66.03 ± 5.62	0.0938
E6	70.36 ± 2.78	62.76 ± 4.10	68.21 ± 4.48	0.0949
E17	69.93 ± 2.87	61.88 ± 3.14	67.16 ± 3.46	0.0943
E5	69.92 ± 3.18	62.71 ± 3.90	66.64 ± 4.19	0.0965
E15	69.56 ± 4.50	58.57 ± 6.35	66.73 ± 4.12	0.0938
E10	69.50 ± 2.53	57.61 ± 2.68	66.64 ± 3.22	0.0928
E2	69.12 ± 4.45	63.58 ± 4.47	67.66 ± 5.92	0.1137
E13	68.93 ± 3.47	58.60 ± 4.32	65.40 ± 4.55	0.0971
E11	68.05 ± 3.58	57.06 ± 6.66	64.88 ± 4.30	0.0973
E14	67.67 ± 2.82	56.75 ± 3.22	64.78 ± 3.72	0.0981
E16	67.48 ± 2.02	53.74 ± 3.49	65.33 ± 2.54	0.0975
E7	67.36 ± 2.08	61.60 ± 2.67	69.53 ± 3.88	0.1352

6.4 Stage 2: Fixed-Split Validation and Test Results

Final-test ordering changes. Python E15 reaches 65.41%, 4.67 percentage points above E1. Java E17 reaches 70.34%, 0.88 points above E1. Pharo E1 remains strongest at 73.71%, and every tested modification scores lower in micro F1. Figure 6.2 and Table 6.5 show these changes without treating them as a new test-driven selection procedure.

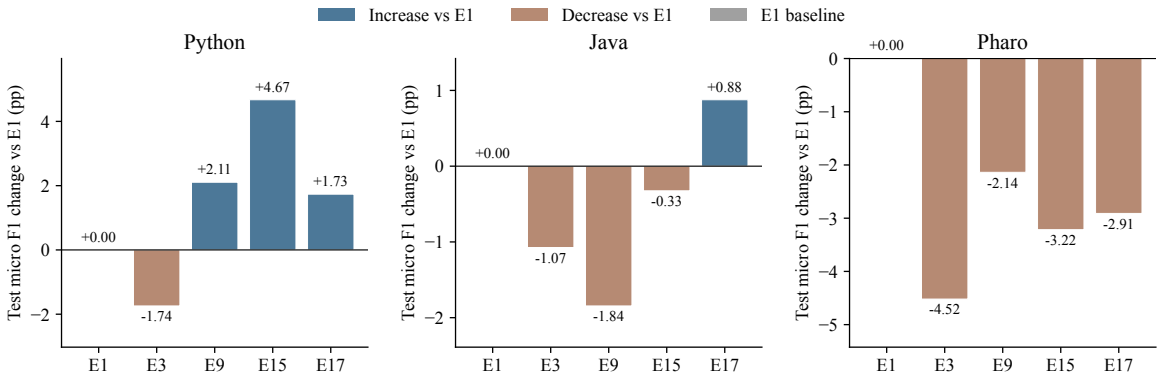


Figure 6.2: Exploratory test micro F1 changes versus E1 (pp). Blue: increase; brown: decrease; gray: baseline.

Table 6.5: Stage-2 checkpoint selection and exploratory test changes versus E1.

Language	ID	Val F1 (%)	Test F1 (%)	Δ (pp)	Epoch	Minutes
Python	E1	63.97	60.74	+0.00	9	3.7
Python	E3	63.62	59.00	-1.74	10	3.7
Python	E9	63.02	62.85	+2.11	6	3.3
Python	E15	65.41	65.41	+4.67	9	6.8
Python	E17	61.33	62.48	+1.73	4	2.3
Java	E1	57.61	69.47	+0.00	6	2.8
Java	E3	60.55	68.39	-1.07	5	2.5
Java	E9	59.65	67.62	-1.84	8	3.4
Java	E15	64.02	69.14	-0.33	3	3.2
Java	E17	63.38	70.34	+0.88	3	1.8
Pharo	E1	77.35	73.71	+0.00	8	2.6
Pharo	E3	76.80	69.19	-4.52	5	1.8
Pharo	E9	73.63	71.57	-2.14	7	2.6
Pharo	E15	75.66	70.50	-3.22	6	3.8
Pharo	E17	71.66	70.80	-2.91	10	2.7

Tables 6.6–6.8 report the five stage-2 systems separately. Test ordering is de-

scriptive; checkpoint selection uses validation.

Table 6.6: Python: stage-2 exploratory test only; ordered by test micro F1 for descriptive reporting.

ID	Test micro F1	Test macro F1	Test macro AP	Test HL
E15	65.41	59.65	70.27	0.1437
E9	62.85	56.34	62.72	0.1570
E17	62.48	56.69	69.29	0.1492
E1	60.74	55.47	61.96	0.1656
E3	59.00	54.96	64.23	0.1727

Table 6.7: Java: stage-2 exploratory test only; ordered by test micro F1 for descriptive reporting.

ID	Test micro F1	Test macro F1	Test macro AP	Test HL
E17	70.34	73.42	81.28	0.0921
E1	69.47	69.46	77.08	0.0945
E15	69.14	72.84	81.17	0.0933
E3	68.39	71.11	81.14	0.1009
E9	67.62	73.71	78.66	0.1068

Table 6.8: Pharo: stage-2 exploratory test only; ordered by test micro F1 for descriptive reporting.

ID	Test micro F1	Test macro F1	Test macro AP	Test HL
E1	73.71	67.20	66.81	0.0828
E9	71.57	60.58	66.93	0.0909
E17	70.80	62.83	70.63	0.0917
E15	70.50	61.94	69.34	0.0917
E3	69.19	60.15	67.42	0.0990

6.5 Ablation Studies

6.5.1 Loss and Attention Contrasts

E2 versus E1 illustrates the difference between micro and macro objectives. Python CV macro F1 increases from 56.12% to 58.34%, while micro F1 changes from 61.69% to 61.61%. Pharo macro F1 also increases, 60.53% to 63.58%, while micro F1 falls to 69.12%. Positive weighting therefore shifts the operating point and label balance rather than uniformly improving every metric.

The E5-to-E6 comparison isolates adding the implemented sentence–label contrastive term to the attention family. CV micro F1 changes from 60.80 to 61.58 in Python, 60.55 to 61.15 in Java, and 69.92 to 70.36 in Pharo. These small positive changes do not imply that either method beats the simpler baseline everywhere. E4 cannot serve as an empirical ablation of threshold tuning because its thresholds remain at 0.5 during screening.

6.5.2 Consistency and Optimization Contrasts

Compared with CodeBERT BCE (E11), R-Drop (E15) improves CV micro F1 in all three languages: 63.97 to 64.79, 59.80 to 62.25, and 68.05 to 69.56. However, E16 does not improve upon E15: adding the head-rate/LLRD configuration yields 63.76, 59.79, and 67.48. The results caution against assuming that individually motivated improvements are additive.

These contrasts are observational ablations under matched splits, not a complete factorial design. E9 differs in stopping policy, additional models have unequal effective epochs, and no repeated-seed uncertainty is available. Figure 6.3 shows the actual final-stage histories. Comparing losses across different objective functions is inappropriate; curves are useful for diagnosing progress within a method, while validation F1 supplies a common operational scale.

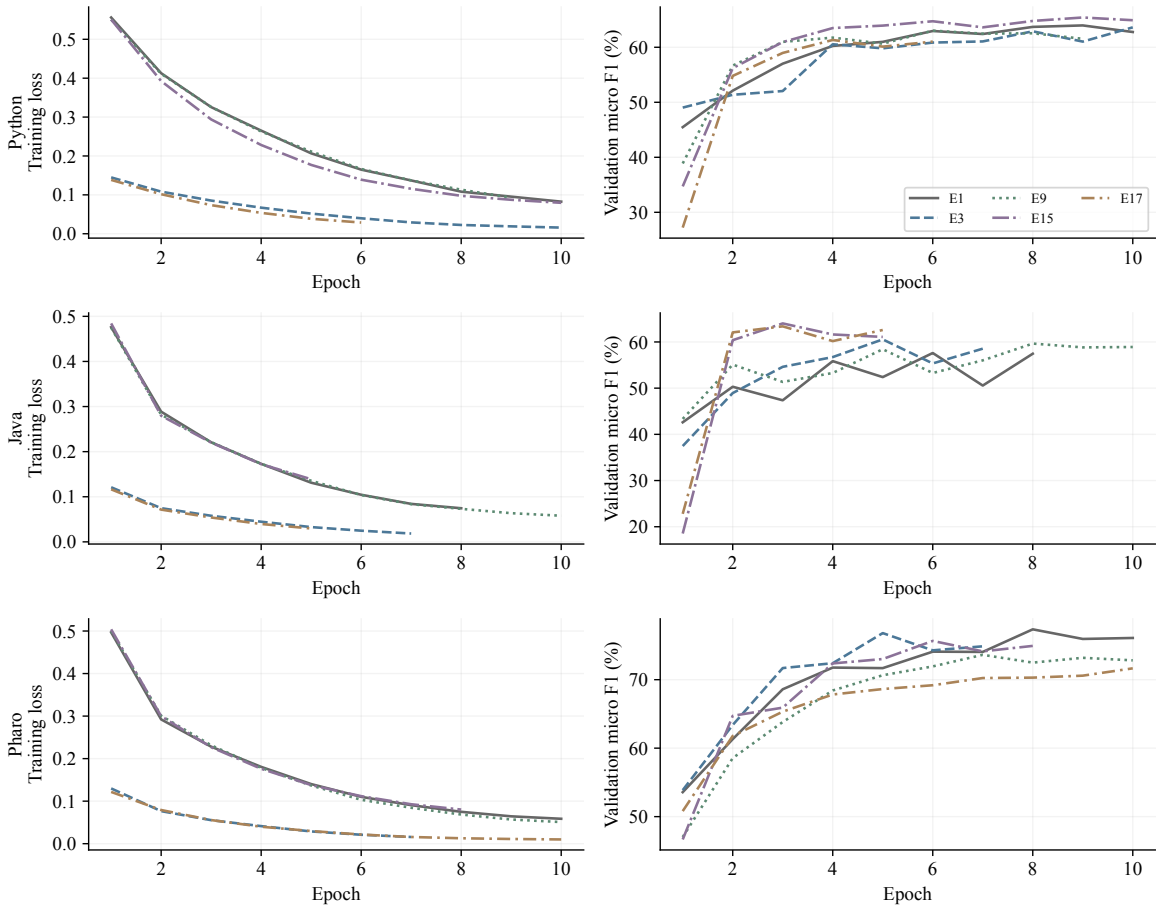


Figure 6.3: Measured final-stage training loss and validation micro F1 by epoch; objectives differ across methods.

6.6 Per-label Performance Analysis

Per-label results reveal behavior hidden by micro aggregation. For Java rational, E1 has recall 10.71% and F1 18.18% on 28 test positives. E9 raises recall to 60.71% and F1 to 72.34%, even though its overall Java micro F1 is below E1. Its macro F1 is 73.71%, exceeding E1's 69.46%. This is a concrete example of minority-label improvement coexisting with a weaker pooled score.

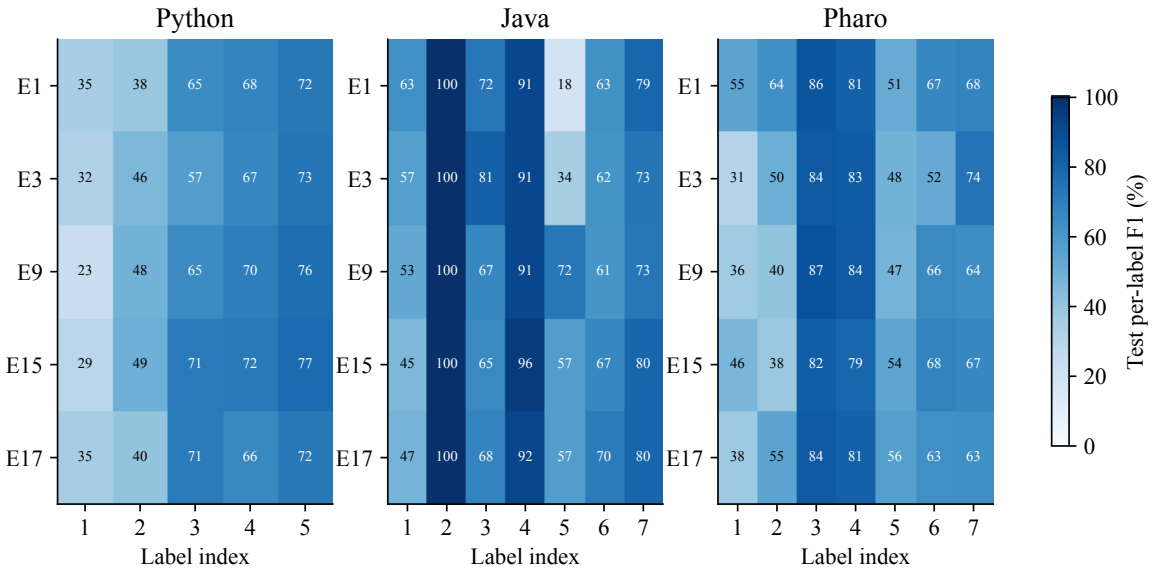


Figure 6.4: Observed test F1 by label and method; darker blue means higher F1. Label indices follow Table 3.3.

Figure 6.4 should be interpreted together with label support, not as a support-free ranking. Pharo Classreferences has only seven final-test positives, so one corrected decision can materially change recall. Appendix C reports all 95 method–label combinations, including precision, recall, AP, and derived false-positive/false-negative counts. No per-label confidence intervals are claimed from these small fixed samples.

6.7 Error Analysis

Table 6.9 reconstructs aggregate confusion counts directly from saved metrics. Python E15 reduces false positives from E1's 89 to 71 and false negatives from 123 to 113. In Pharo, E3 has 55 false positives and 67 false negatives, compared with E1's 41 and 61; both error types worsen. Java E9 gains true positives but also produces more false positives, illustrating a different operating trade-off rather than a uniform improvement.

Table 6.9: Observed test confusion counts aggregated across labels.

Language	ID	TP	FP	FN	TN	P (%)	R (%)
Python	E1	164	89	123	904	64.82	57.14
Python	E3	159	93	128	900	63.10	55.40
Python	E9	170	84	117	909	66.93	59.23
Python	E15	174	71	113	922	71.02	60.63
Python	E17	159	63	128	930	71.62	55.40
Java	E1	182	60	100	1352	75.21	64.54
Java	E3	185	74	97	1338	71.43	65.60
Java	E9	189	88	93	1324	68.23	67.02
Java	E15	177	53	105	1359	76.96	62.77
Java	E17	185	59	97	1353	75.82	65.60
Pharo	E1	143	41	61	987	77.72	70.10
Pharo	E3	137	55	67	973	71.35	67.16
Pharo	E9	141	49	63	979	74.21	69.12
Pharo	E15	135	44	69	984	75.42	66.18
Pharo	E17	137	46	67	982	74.86	67.16

The current files do not retain the mapping from sentence ID to predicted probability. Consequently, it is not possible to identify specific misclassified sentences, measure error overlap between models, calculate calibration curves, or evaluate an ensemble without new inference. Explanations involving negation, identifier ambiguity, or missing context remain hypotheses. The report does not present invented examples as observed errors.

6.8 Discussion of Research Questions

RQ1 receives a conditional answer: CodeBERT is competitive in Python and supports strong Java variants, but does not dominate Pharo. RQ2 is metric dependent: weighting helps macro F1 in several cases, attention/contrastive changes are modest, and CodeBERT R-Drop improves screening scores over its BCE baseline. RQ3 is answered negatively by the different stage-1 and stage-2 winners. RQ4 shows that rare-label recall and pooled F1 can move in opposite directions.

No tested single method is best across all three programming languages. This motivates a validation-selected ensemble or an adaptive model-selection procedure, but their effectiveness is not yet measured. Simply combining the strongest test models would incorporate test information into a new development decision. Any such extension should freeze its selection rule on validation or nested training folds and be

reported as exploratory until independently confirmed.

6.9 Limitations and Threats to Validity

Internal validity is limited by full-population screening, validation reuse for epoch selection, E9's distinct stopping policy, and single-seed final training. Construct validity is limited by the gap between sentence labels and practical documentation quality, the difference between thresholded and ranking metrics, and the absence of a comparable ML baseline. External validity is constrained by shared classes across splits, unresolved near-duplicates, and only three programming ecosystems.

Computational comparisons are also limited: recorded durations depend on device, concurrent load, stopping time, and objective cost. There is no controlled inference-latency or energy benchmark. Model rankings reflect finite observations and rounding can conceal small differences. No significance test, causal superiority, or official competition victory is claimed. The most important corrective experiment is an untouched project/group holdout established before further method selection.

CHAPTER 7. CONCLUSION AND FUTURE WORK

7.1 Summary of the Research

This study organizes code-comment classification as a language-specific multi-label problem and audits a two-stage experimental workflow. It evaluates seventeen Transformer configurations through five-fold screening and five selected methods through fixed 80/10/10 training. The report provides verified aggregate, fold-level, and per-label measurements while preserving the distinction between measured systems, implemented but unevaluated ML code, and proposed future baselines.

7.2 Main Findings

The strongest observed stage-2 micro F1 values are 65.41% for Python E15, 70.34% for Java E17, and 73.71% for Pharo E1. No universal winning model emerges. Imbalance handling can improve macro F1 without improving micro F1, and graph modeling can recover rare-label positives while increasing false positives elsewhere. More complex combinations, exemplified by E16, do not automatically outperform simpler variants under the recorded settings.

7.3 Research Limitations

The final test population is not independent of stage-1 method selection. All conclusions therefore concern exploratory comparisons rather than an unbiased estimate of the selected pipeline's future performance. Missing comparable ML runs, historical environment locks, repeated seeds, and sentence-level predictions further limit the scope of inference. These gaps are reported explicitly instead of being hidden by a complete-looking numerical table.

7.4 Future Research Directions

The first priority is a confirmatory design with test projects or groups reserved before any screening. Nested grouped CV and repeated base seeds should then separate selection effects from training variability. Classical TF-IDF baselines should be integrated into the same loader and metric implementation. Recording probabilities by sentence would enable paired error analysis, calibration, and transparent threshold experiments.

A second direction is to combine complementary pretrained encoders through

a small validation-tuned probability ensemble, following the general ensemble principle [21]. A common architecture can retain language-specific weights without assuming identical optimal decision rules. Context from surrounding comments or source code and explicit annotation-ambiguity analysis are further candidates. These are research proposals; the current evidence does not show that any one extension will lead every language.

BIBLIOGRAPHY

[1] NLBSE 2023 Organizers, "NLBSE'23 tool competition: Code comment classification." `https://nlbse2023.github.io/tools/`, 2023. Accessed November 3, 2026.

[2] NLBSE 2023 Organizers, "Code-comment-classification: Dataset, preparation, and baseline classifiers." `https://github.com/nlbse2023/code-comment-classification`, 2023. Official competition repository; accessed November 3, 2026.

[3] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[4] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[7] J. Read, B. Pfahringer, G. Holmes, and E. Frank, "Classifier chains for multi-label classification," in *Machine Learning and Knowledge Discovery in Databases*, pp. 254–269, 2009.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[9] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[10] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020.

[11] M.-L. Zhang and Z.-H. Zhou, "A review on multi-label learning algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014.

[12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2980–2988, 2017.

[13] T. Ridnik, E. Ben-Baruch, N. Zamir, A. Noy, I. Friedman, M. Protter, and L. Zelnik-Manor, "Asymmetric loss for multi-label classification," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 82–91, 2021.

[14] J. Mullenbach, S. Wiegreffe, J. Duke, J. Sun, and J. Eisenstein, "Explainable prediction of medical codes from clinical text," in *Proceedings of NAACL-HLT*, pp. 1101–1111, 2018.

[15] X. Liang, L. Wu, J. Li, Y. Wang, Q. Meng, T. Qin, W. Chen, M. Zhang, and T.-Y. Liu, "R-Drop: Regularized dropout for neural networks," in *Advances in Neural Information Processing Systems*, vol. 34, pp. 10890–10905, 2021.

[16] Z.-M. Chen, X.-S. Wei, P. Wang, and Y. Guo, "Multi-label image recognition with graph convolutional networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5177–5186, 2019.

[17] T. Miyato, A. M. Dai, and I. Goodfellow, "Adversarial training methods for semi-supervised text classification," in *International Conference on Learning Representations*, 2017.

[18] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 18661–18673, 2020.

[19] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[20] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, p. 91, 2006.

[21] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

APPENDIX A. HYPERPARAMETER CONFIGURATIONS

Table A.1 summarizes the values recorded in effective run configurations. Dataset paths inside default YAML files can refer to legacy data, but staged runners override those paths with the selected stage directory. Saved run configs and fingerprints establish which data were actually used.

Table A.1: Recorded hyperparameters; overrides are preserved in each run config.

Setting	Value
Backbone	roberta-base / microsoft/codebert-base
Maximum length	128
Train / evaluation batch	16 / 32
Maximum epochs	10
Base learning rate	2×10^{-5}
AdamW weight decay	0.01
Warmup ratio	0.1
Gradient clipping	1.0
Mixed precision	False
Default patience	2
E9 patience / minimum epochs	3 / 5
E9 tie-break	Micro AP
Focal gamma / alpha	2 / none
E6 contrastive weight / temperature	0.1 / 0.1
ASL positive / negative gamma / clip	1 / 4 / 0.05
R-Drop alpha / FGM epsilon	1 / 1
Graph hidden size / threshold	128 / 0.1
E13–E16 head LR where configured	10^{-4} (E13, E14, E16)
LLRD factor	0.9 (E14, E16)

APPENDIX B. DETAILED CROSS-VALIDATION RESULTS

Table B.1 lists best-checkpoint validation results for every fold of every completed screening run. Percentages are used for F1 and AP; Hamming loss remains a proportion. Fold labels are one-based for readability, whereas the implementation stores zero-based indices.

Table B.1: All 255 fold-level best-checkpoint validation results.

Language	ID	Fold	Epoch	Micro F1	Macro F1	Macro AP	HL
Python	E1	1	10	59.66	54.31	61.29	0.1742
Python	E1	2	8	61.28	55.56	64.20	0.1652
Python	E1	3	9	63.56	57.83	66.31	0.1562
Python	E1	4	10	63.50	58.07	62.37	0.1596
Python	E1	5	8	60.46	54.81	62.10	0.1676
Python	E2	1	9	58.70	56.87	62.97	0.1933
Python	E2	2	9	61.74	58.86	63.23	0.1843
Python	E2	3	6	63.58	60.13	63.77	0.1753
Python	E2	4	7	63.26	59.53	62.29	0.1776
Python	E2	5	10	60.77	56.30	62.93	0.1801
Python	E3	1	10	58.72	52.63	62.93	0.1695
Python	E3	2	7	61.70	55.12	64.51	0.1609
Python	E3	3	9	64.64	58.94	67.68	0.1503
Python	E3	4	8	64.78	59.80	65.74	0.1514
Python	E3	5	5	61.10	56.16	63.09	0.1656
Python	E4	1	10	59.66	54.31	61.29	0.1742
Python	E4	2	8	61.28	55.56	64.20	0.1652
Python	E4	3	9	63.56	57.83	66.31	0.1562
Python	E4	4	10	63.50	58.07	62.37	0.1596
Python	E4	5	8	60.46	54.81	62.10	0.1676
Python	E5	1	6	58.33	53.66	64.38	0.1734
Python	E5	2	8	62.01	55.02	62.73	0.1616
Python	E5	3	9	62.79	57.11	64.91	0.1628
Python	E5	4	10	62.21	57.90	63.12	0.1620
Python	E5	5	4	58.64	48.86	61.07	0.1664
Python	E6	1	6	58.64	52.68	61.75	0.1695
Python	E6	2	8	62.60	59.07	64.83	0.1632
Python	E6	3	8	62.59	55.50	64.67	0.1605
Python	E6	4	6	62.27	54.15	62.71	0.1549
Python	E6	5	7	61.79	56.77	62.72	0.1613
Python	E7	1	10	60.85	58.10	63.43	0.2196
Python	E7	2	7	60.01	57.58	65.92	0.2258
Python	E7	3	10	61.71	58.65	67.44	0.2098
Python	E7	4	10	61.89	58.92	66.88	0.2149
Python	E7	5	8	59.57	56.87	64.50	0.2211
Python	E8	1	7	59.54	53.17	62.37	0.1718

Language	ID	Fold	Epoch	Micro F1	Macro F1	Macro AP	HL
Python	E8	2	8	63.32	56.39	65.67	0.1515
Python	E8	3	9	66.61	61.80	69.02	0.1417
Python	E8	4	5	63.79	56.74	63.63	0.1447
Python	E8	5	9	60.31	56.02	63.46	0.1676
Python	E9	1	9	57.93	53.92	63.15	0.1796
Python	E9	2	7	63.27	59.40	64.36	0.1581
Python	E9	3	8	65.15	58.97	65.55	0.1499
Python	E9	4	8	62.67	56.55	62.42	0.1635
Python	E9	5	10	61.51	57.32	62.65	0.1652
Python	E10	1	8	62.98	53.15	64.39	0.1495
Python	E10	2	8	62.28	52.78	63.77	0.1503
Python	E10	3	7	63.64	53.38	65.88	0.1440
Python	E10	4	9	64.66	56.00	63.46	0.1427
Python	E10	5	10	59.46	52.82	63.91	0.1656
Python	E11	1	6	66.35	59.50	68.11	0.1401
Python	E11	2	6	61.94	55.80	66.01	0.1534
Python	E11	3	4	63.69	52.53	66.41	0.1464
Python	E11	4	8	65.20	60.91	65.39	0.1486
Python	E11	5	7	62.67	58.27	64.26	0.1582
Python	E12	1	8	59.49	55.47	64.77	0.1753
Python	E12	2	6	61.24	56.15	63.11	0.1659
Python	E12	3	5	60.04	51.75	63.98	0.1620
Python	E12	4	10	64.14	60.38	64.43	0.1522
Python	E12	5	7	61.67	56.11	63.63	0.1637
Python	E13	1	6	66.23	60.19	67.86	0.1417
Python	E13	2	8	63.66	58.45	65.20	0.1546
Python	E13	3	8	65.18	57.60	68.16	0.1464
Python	E13	4	9	65.50	61.01	66.69	0.1471
Python	E13	5	7	62.16	57.36	64.05	0.1598
Python	E14	1	6	63.91	54.29	66.21	0.1468
Python	E14	2	7	63.95	57.96	65.93	0.1456
Python	E14	3	9	63.38	53.89	66.07	0.1511
Python	E14	4	7	64.50	57.53	62.48	0.1455
Python	E14	5	6	59.63	50.35	61.76	0.1629
Python	E15	1	6	68.01	60.76	69.62	0.1307
Python	E15	2	9	63.86	57.72	65.78	0.1511
Python	E15	3	5	64.35	57.38	68.23	0.1444
Python	E15	4	7	66.05	61.20	66.77	0.1431
Python	E15	5	6	61.70	55.98	64.73	0.1586
Python	E16	1	6	65.00	53.86	67.18	0.1370
Python	E16	2	10	65.06	58.87	67.15	0.1417
Python	E16	3	7	63.73	55.17	67.90	0.1452
Python	E16	4	9	64.23	57.40	64.74	0.1459
Python	E16	5	8	60.80	53.76	63.43	0.1566
Python	E17	1	6	66.36	61.92	70.16	0.1440
Python	E17	2	6	65.36	60.54	67.97	0.1452
Python	E17	3	5	63.98	57.02	68.28	0.1472
Python	E17	4	8	65.93	61.56	65.56	0.1443

Language	ID	Fold	Epoch	Micro F1	Macro F1	Macro AP	HL
Python	E17	5	6	60.95	55.87	64.79	0.1637
Java	E1	1	8	62.55	63.28	67.77	0.1219
Java	E1	2	8	68.92	73.15	76.82	0.1030
Java	E1	3	10	61.24	66.27	68.26	0.1237
Java	E1	4	9	52.90	55.39	59.70	0.1491
Java	E1	5	3	52.82	55.56	64.32	0.1508
Java	E2	1	6	60.83	63.45	66.34	0.1417
Java	E2	2	5	68.15	73.70	77.86	0.1228
Java	E2	3	10	61.80	66.75	66.38	0.1328
Java	E2	4	2	53.22	57.64	56.33	0.2017
Java	E2	5	3	55.03	63.08	63.93	0.1924
Java	E3	1	6	60.80	63.57	68.93	0.1213
Java	E3	2	6	68.39	73.39	77.86	0.1012
Java	E3	3	5	59.29	62.85	66.40	0.1293
Java	E3	4	8	52.28	54.17	58.85	0.1517
Java	E3	5	6	56.93	61.62	64.16	0.1349
Java	E4	1	8	62.55	63.28	67.77	0.1219
Java	E4	2	8	68.92	73.15	76.82	0.1030
Java	E4	3	10	61.24	66.27	68.26	0.1237
Java	E4	4	9	52.90	55.39	59.70	0.1491
Java	E4	5	3	52.82	55.56	64.32	0.1508
Java	E5	1	3	62.14	65.81	67.93	0.1204
Java	E5	2	5	66.92	66.60	76.55	0.1042
Java	E5	3	5	59.33	64.08	67.39	0.1254
Java	E5	4	3	53.44	56.98	61.42	0.1402
Java	E5	5	8	60.93	66.85	69.93	0.1260
Java	E6	1	9	63.21	66.00	65.86	0.1239
Java	E6	2	7	69.39	74.31	77.93	0.0995
Java	E6	3	6	60.20	64.48	66.03	0.1272
Java	E6	4	2	50.46	53.90	60.60	0.1446
Java	E6	5	8	62.47	69.17	69.72	0.1213
Java	E7	1	5	60.74	63.97	69.22	0.1801
Java	E7	2	8	67.32	70.21	78.23	0.1372
Java	E7	3	8	58.92	63.50	66.85	0.1638
Java	E7	4	9	51.82	55.64	59.43	0.1958
Java	E7	5	6	57.12	63.52	65.04	0.1927
Java	E8	1	2	58.72	59.27	66.24	0.1281
Java	E8	2	5	69.78	74.03	78.14	0.0953
Java	E8	3	6	59.20	60.03	68.01	0.1257
Java	E8	4	6	56.60	60.00	62.78	0.1352
Java	E8	5	10	58.94	64.73	66.91	0.1328
Java	E9	1	7	63.41	64.27	68.94	0.1174
Java	E9	2	7	68.76	73.67	75.79	0.1001
Java	E9	3	10	61.62	66.09	67.90	0.1228
Java	E9	4	3	53.80	53.27	58.50	0.1473
Java	E9	5	5	58.65	66.17	65.25	0.1340
Java	E10	1	5	61.13	66.04	70.66	0.1245
Java	E10	2	6	66.60	72.05	76.12	0.1051

Language	ID	Fold	Epoch	Micro F1	Macro F1	Macro AP	HL
Java	E10	3	9	57.78	60.34	65.29	0.1337
Java	E10	4	6	54.29	54.59	63.39	0.1369
Java	E10	5	9	58.03	63.17	65.60	0.1319
Java	E11	1	5	63.09	63.13	71.18	0.1159
Java	E11	2	7	62.11	69.63	74.95	0.1228
Java	E11	3	6	59.25	61.82	66.67	0.1275
Java	E11	4	3	55.45	55.89	69.45	0.1245
Java	E11	5	3	59.10	62.65	67.70	0.1201
Java	E12	1	8	62.69	64.24	66.34	0.1278
Java	E12	2	8	68.12	71.32	76.16	0.1039
Java	E12	3	9	60.07	60.89	65.83	0.1299
Java	E12	4	7	53.68	59.70	61.81	0.1491
Java	E12	5	7	61.29	68.68	68.38	0.1260
Java	E13	1	8	62.98	64.57	69.81	0.1168
Java	E13	2	4	59.87	66.37	73.59	0.1231
Java	E13	3	7	60.54	65.75	66.11	0.1243
Java	E13	4	3	55.38	57.09	68.18	0.1239
Java	E13	5	7	61.29	67.31	65.90	0.1275
Java	E14	1	5	62.71	66.52	71.79	0.1115
Java	E14	2	5	60.13	65.98	73.51	0.1249
Java	E14	3	6	58.97	62.34	66.31	0.1290
Java	E14	4	6	57.79	59.69	68.95	0.1266
Java	E14	5	6	62.02	67.41	68.30	0.1175
Java	E15	1	7	64.30	65.47	71.04	0.1159
Java	E15	2	4	67.68	71.69	76.83	0.1006
Java	E15	3	5	59.70	61.99	65.77	0.1275
Java	E15	4	3	56.88	57.68	65.74	0.1224
Java	E15	5	7	62.70	67.93	65.95	0.1184
Java	E16	1	4	64.90	67.69	72.99	0.1094
Java	E16	2	4	61.01	67.27	76.10	0.1166
Java	E16	3	4	59.13	58.24	66.46	0.1216
Java	E16	4	3	53.71	56.82	65.91	0.1275
Java	E16	5	10	60.22	64.26	67.50	0.1201
Java	E17	1	5	62.85	67.37	71.09	0.1133
Java	E17	2	8	63.42	70.08	74.61	0.1175
Java	E17	3	5	61.85	65.82	69.26	0.1231
Java	E17	4	4	57.03	59.02	67.46	0.1275
Java	E17	5	5	62.56	62.99	67.56	0.1219
Pharo	E1	1	9	69.80	57.83	63.53	0.0966
Pharo	E1	2	9	75.53	67.44	74.03	0.0787
Pharo	E1	3	8	66.75	58.00	62.90	0.1048
Pharo	E1	4	8	72.86	64.40	69.53	0.0874
Pharo	E1	5	9	67.28	55.00	60.17	0.1016
Pharo	E2	1	10	69.54	61.40	64.97	0.1092
Pharo	E2	2	10	73.97	69.41	75.04	0.0940
Pharo	E2	3	6	62.78	58.39	64.60	0.1420
Pharo	E2	4	10	72.39	66.89	72.67	0.0975
Pharo	E2	5	6	66.95	61.79	61.03	0.1259

Language	ID	Fold	Epoch	Micro F1	Macro F1	Macro AP	HL
Pharo	E3	1	7	70.39	58.64	64.07	0.0925
Pharo	E3	2	8	73.99	67.59	73.04	0.0831
Pharo	E3	3	9	68.58	60.16	69.65	0.0975
Pharo	E3	4	9	73.82	65.37	70.89	0.0850
Pharo	E3	5	7	70.82	61.61	62.53	0.0947
Pharo	E4	1	9	69.80	57.83	63.53	0.0966
Pharo	E4	2	9	75.53	67.44	74.03	0.0787
Pharo	E4	3	8	66.75	58.00	62.90	0.1048
Pharo	E4	4	8	72.86	64.40	69.53	0.0874
Pharo	E4	5	9	67.28	55.00	60.17	0.1016
Pharo	E5	1	9	72.68	65.18	68.27	0.0885
Pharo	E5	2	7	71.86	67.18	71.76	0.0904
Pharo	E5	3	4	65.06	57.58	61.06	0.1113
Pharo	E5	4	9	71.63	63.62	68.26	0.0911
Pharo	E5	5	7	68.35	59.97	63.87	0.1012
Pharo	E6	1	10	71.45	62.37	65.64	0.0921
Pharo	E6	2	7	72.18	65.48	72.99	0.0896
Pharo	E6	3	9	66.58	58.00	66.77	0.1081
Pharo	E6	4	7	73.24	68.10	72.76	0.0846
Pharo	E6	5	10	68.37	59.84	62.90	0.1004
Pharo	E7	1	10	65.80	58.95	65.46	0.1384
Pharo	E7	2	7	68.85	63.04	73.41	0.1303
Pharo	E7	3	6	65.74	60.96	69.96	0.1489
Pharo	E7	4	10	70.27	65.48	73.18	0.1202
Pharo	E7	5	9	66.14	59.58	65.66	0.1380
Pharo	E8	1	6	70.88	60.47	66.23	0.0917
Pharo	E8	2	5	74.41	65.31	73.65	0.0783
Pharo	E8	3	7	67.27	58.37	65.03	0.1032
Pharo	E8	4	7	73.78	64.04	70.98	0.0826
Pharo	E8	5	6	68.89	55.38	63.48	0.0943
Pharo	E9	1	10	69.43	58.06	60.73	0.0986
Pharo	E9	2	10	74.94	67.22	73.34	0.0799
Pharo	E9	3	7	71.26	62.16	67.57	0.0894
Pharo	E9	4	8	73.29	62.23	68.89	0.0838
Pharo	E9	5	9	67.27	55.02	62.40	0.1028
Pharo	E10	1	7	71.80	58.68	64.73	0.0877
Pharo	E10	2	5	72.22	60.46	71.69	0.0807
Pharo	E10	3	7	66.75	57.81	65.74	0.1024
Pharo	E10	4	6	69.52	53.20	67.64	0.0923
Pharo	E10	5	10	67.19	57.88	63.41	0.1012
Pharo	E11	1	10	70.48	60.79	66.02	0.0942
Pharo	E11	2	7	72.37	63.41	69.27	0.0847
Pharo	E11	3	8	66.50	57.02	63.31	0.1060
Pharo	E11	4	7	67.74	58.07	67.55	0.0971
Pharo	E11	5	3	63.14	46.00	58.27	0.1044
Pharo	E12	1	8	72.16	64.42	65.54	0.0905
Pharo	E12	2	10	74.53	68.81	69.57	0.0827
Pharo	E12	3	4	65.55	54.38	63.52	0.1085

Language	ID	Fold	Epoch	Micro F1	Macro F1	Macro AP	HL
Pharo	E12	4	10	72.89	66.27	71.76	0.0870
Pharo	E12	5	9	68.29	58.36	61.93	0.1004
Pharo	E13	1	4	69.09	54.04	62.47	0.0933
Pharo	E13	2	10	74.15	65.33	71.84	0.0827
Pharo	E13	3	8	65.22	56.46	62.84	0.1101
Pharo	E13	4	10	69.77	59.99	68.51	0.0947
Pharo	E13	5	9	66.41	57.20	61.34	0.1044
Pharo	E14	1	7	67.90	56.77	64.75	0.0982
Pharo	E14	2	8	72.10	61.97	70.82	0.0843
Pharo	E14	3	8	64.69	54.25	62.62	0.1109
Pharo	E14	4	6	67.78	53.94	64.73	0.0935
Pharo	E14	5	7	65.87	56.81	61.00	0.1036
Pharo	E15	1	9	71.28	60.98	66.35	0.0909
Pharo	E15	2	9	76.35	67.71	73.43	0.0743
Pharo	E15	3	5	65.26	50.65	63.96	0.1068
Pharo	E15	4	7	68.97	57.89	67.10	0.0943
Pharo	E15	5	6	65.95	55.61	62.83	0.1028
Pharo	E16	1	8	69.82	57.01	65.57	0.0933
Pharo	E16	2	7	69.41	56.07	69.29	0.0900
Pharo	E16	3	9	65.45	50.04	63.76	0.1068
Pharo	E16	4	6	66.86	49.87	65.47	0.0947
Pharo	E16	5	7	65.86	55.71	62.57	0.1028
Pharo	E17	1	7	70.33	61.46	66.16	0.0942
Pharo	E17	2	10	74.71	67.16	72.29	0.0803
Pharo	E17	3	10	67.43	61.75	66.45	0.1032
Pharo	E17	4	9	68.50	59.69	68.12	0.0971
Pharo	E17	5	10	68.68	59.33	62.77	0.0967

APPENDIX C. PER-LABEL EVALUATION RESULTS

Table C.1 reports test support, precision, recall, F1, AP, false positives, and false negatives. These are one-seed observations. FP equals predicted positives minus true positives; FN equals support minus true positives.

Table C.1: Complete stage-2 test per-label results (95 model–label combinations).

Language	ID	Label	n	P	R	F1	AP	FP	FN
Python	E1	DevelopmentNotes	31	42.86	29.03	34.62	39.98	12	22
Python	E1	Expand	51	55.56	29.41	38.46	49.59	12	36
Python	E1	Parameters	80	64.20	65.00	64.60	67.32	29	28
Python	E1	Summary	45	77.14	60.00	67.50	71.98	8	18
Python	E1	Usage	80	68.54	76.25	72.19	80.94	28	19
Python	E3	DevelopmentNotes	31	42.11	25.81	32.00	40.99	11	23
Python	E3	Expand	51	51.22	41.18	45.65	50.16	20	30
Python	E3	Parameters	80	59.46	55.00	57.14	73.95	30	36
Python	E3	Summary	45	64.58	68.89	66.67	72.09	17	14
Python	E3	Usage	80	78.57	68.75	73.33	83.97	15	25
Python	E9	DevelopmentNotes	31	41.67	16.13	23.26	33.70	7	26
Python	E9	Expand	51	56.76	41.18	47.73	51.03	16	30
Python	E9	Parameters	80	63.86	66.25	65.03	72.94	30	27
Python	E9	Summary	45	68.09	71.11	69.57	72.81	15	13
Python	E9	Usage	80	78.67	73.75	76.13	83.13	16	21
Python	E15	DevelopmentNotes	31	38.89	22.58	28.57	42.55	11	24
Python	E15	Expand	51	57.89	43.14	49.44	57.68	16	29
Python	E15	Parameters	80	74.32	68.75	71.43	81.11	19	25
Python	E15	Summary	45	68.00	75.56	71.58	81.34	16	11
Python	E15	Usage	80	86.15	70.00	77.24	88.68	9	24
Python	E17	DevelopmentNotes	31	53.33	25.81	34.78	43.80	7	23
Python	E17	Expand	51	62.50	29.41	40.00	54.60	9	36
Python	E17	Parameters	80	70.37	71.25	70.81	83.46	24	23
Python	E17	Summary	45	67.44	64.44	65.91	78.18	14	16
Python	E17	Usage	80	84.75	62.50	71.94	86.39	9	30
Java	E1	Expand	63	75.56	53.97	62.96	68.33	11	29
Java	E1	Ownership	11	100.00	100.00	100.00	100.00	0	0
Java	E1	Pointer	35	91.30	60.00	72.41	81.80	2	14
Java	E1	deprecation	12	100.00	83.33	90.91	91.20	0	2
Java	E1	rational	28	60.00	10.71	18.18	41.77	2	25
Java	E1	summary	42	54.39	73.81	62.63	67.85	26	11
Java	E1	usage	91	79.12	79.12	79.12	88.64	19	19
Java	E3	Expand	63	64.00	50.79	56.64	67.95	18	31
Java	E3	Ownership	11	100.00	100.00	100.00	100.00	0	0
Java	E3	Pointer	35	82.35	80.00	81.16	87.22	6	7
Java	E3	deprecation	12	100.00	83.33	90.91	91.29	0	2
Java	E3	rational	28	85.71	21.43	34.29	64.05	1	22
Java	E3	summary	42	66.67	57.14	61.54	70.24	12	18
Java	E3	usage	91	66.67	81.32	73.27	87.25	37	17
Java	E9	Expand	63	58.82	47.62	52.63	60.05	21	33
Java	E9	Ownership	11	100.00	100.00	100.00	100.00	0	0
Java	E9	Pointer	35	67.65	65.71	66.67	82.27	11	12
Java	E9	deprecation	12	100.00	83.33	90.91	92.71	0	2
Java	E9	rational	28	89.47	60.71	72.34	65.88	2	11
Java	E9	summary	42	56.00	66.67	60.87	65.33	22	14
Java	E9	usage	91	68.63	76.92	72.54	84.37	32	21
Java	E15	Expand	63	64.71	34.92	45.36	62.38	12	41

Language	ID	Label	n	P	R	F1	AP	FP	FN
Java	E15	Ownership	11	100.00	100.00	100.00	100.00	0	0
Java	E15	Pointer	35	90.00	51.43	65.45	81.69	2	17
Java	E15	deprecation	12	100.00	91.67	95.65	92.23	0	1
Java	E15	rational	28	85.71	42.86	57.14	69.44	2	16
Java	E15	summary	42	64.44	69.05	66.67	75.78	16	13
Java	E15	usage	91	77.89	81.32	79.57	86.69	21	17
Java	E17	Expand	63	67.65	36.51	47.42	63.00	11	40
Java	E17	Ownership	11	100.00	100.00	100.00	100.00	0	0
Java	E17	Pointer	35	83.33	57.14	67.80	80.73	4	15
Java	E17	deprecation	12	91.67	91.67	91.67	91.62	1	1
Java	E17	rational	28	85.71	42.86	57.14	72.08	2	16
Java	E17	summary	42	65.31	76.19	70.33	74.12	17	10
Java	E17	usage	91	76.00	83.52	79.58	87.41	24	15
Pharo	E1	Classreferences	7	75.00	42.86	54.55	37.33	1	4
Pharo	E1	Collaborators	13	77.78	53.85	63.64	47.34	2	6
Pharo	E1	Example	75	83.54	88.00	85.71	94.12	13	9
Pharo	E1	Intent	21	80.95	80.95	80.95	88.49	4	4
Pharo	E1	Keyimplementationpoints	23	55.00	47.83	51.16	51.01	9	12
Pharo	E1	Keymessages	31	85.00	54.84	66.67	74.02	3	14
Pharo	E1	Responsibilities	34	70.97	64.71	67.69	75.39	9	12
Pharo	E3	Classreferences	7	33.33	28.57	30.77	31.73	4	5
Pharo	E3	Collaborators	13	71.43	38.46	50.00	50.67	2	8
Pharo	E3	Example	75	87.14	81.33	84.14	94.07	9	14
Pharo	E3	Intent	21	85.00	80.95	82.93	89.31	3	4
Pharo	E3	Keyimplementationpoints	23	47.83	47.83	47.83	59.55	12	12
Pharo	E3	Keymessages	31	48.57	54.84	51.52	66.46	18	14
Pharo	E3	Responsibilities	34	77.42	70.59	73.85	80.13	7	10
Pharo	E9	Classreferences	7	50.00	28.57	36.36	37.95	2	5
Pharo	E9	Collaborators	13	57.14	30.77	40.00	54.12	3	9
Pharo	E9	Example	75	87.84	86.67	87.25	93.01	9	10
Pharo	E9	Intent	21	81.82	85.71	83.72	89.29	4	3
Pharo	E9	Keyimplementationpoints	23	60.00	39.13	47.37	57.71	6	14
Pharo	E9	Keymessages	31	63.64	67.74	65.62	67.54	12	10
Pharo	E9	Responsibilities	34	62.86	64.71	63.77	68.88	13	12
Pharo	E15	Classreferences	7	50.00	42.86	46.15	54.28	3	4
Pharo	E15	Collaborators	13	50.00	30.77	38.10	46.11	4	9
Pharo	E15	Example	75	84.51	80.00	82.19	94.02	11	15
Pharo	E15	Intent	21	77.27	80.95	79.07	89.05	5	4
Pharo	E15	Keyimplementationpoints	23	61.11	47.83	53.66	55.64	7	12
Pharo	E15	Keymessages	31	67.74	67.74	67.74	69.26	10	10
Pharo	E15	Responsibilities	34	82.61	55.88	66.67	77.04	4	15
Pharo	E17	Classreferences	7	33.33	42.86	37.50	50.03	6	4
Pharo	E17	Collaborators	13	66.67	46.15	54.55	50.90	3	7
Pharo	E17	Example	75	84.93	82.67	83.78	94.59	11	13
Pharo	E17	Intent	21	80.95	80.95	80.95	87.05	4	4
Pharo	E17	Keyimplementationpoints	23	68.75	47.83	56.41	67.93	5	12
Pharo	E17	Keymessages	31	69.23	58.06	63.16	73.24	8	13
Pharo	E17	Responsibilities	34	68.97	58.82	63.49	70.69	9	14

APPENDIX D. ADDITIONAL FIGURES AND UPDATED EDA EVIDENCE

This appendix documents the revised research-oriented EDA. The raw population, model targets, grouped folds, and measured model results are unchanged. Figures are selected to answer distinct questions about support, dependence, grouping, and partition validity. Original category-specific partitions are labeled explicitly and are not identified with the fixed stage-2 split. Tables D.1–D.3 preserve the cross-stage result mapping required for numerical reconciliation.

Table D.1: Python: stage-1 CV and available exploratory stage-2 test scores. F1/AP in percent; – denotes not evaluated in stage 2.

ID	CV micro F1 ± SD	CV macro F1 ± SD	Test micro F1	Test macro F1	Test macro AP	Test HL
E15	64.79 ± 2.37	58.61 ± 2.27	65.41	59.65	70.27	0.1437
E13	64.55 ± 1.63	58.93 ± 1.61	–	–	–	–
E17	64.52 ± 2.19	59.38 ± 2.76	62.48	56.69	69.29	0.1492
E11	63.97 ± 1.81	57.40 ± 3.31	–	–	–	–
E16	63.76 ± 1.75	55.81 ± 2.25	–	–	–	–
E14	63.07 ± 1.96	54.80 ± 3.09	–	–	–	–
E8	62.71 ± 2.85	56.83 ± 3.12	–	–	–	–
E10	62.60 ± 1.96	53.63 ± 1.35	–	–	–	–
E3	62.19 ± 2.56	56.53 ± 2.91	59.00	54.96	64.23	0.1727
E9	62.11 ± 2.68	57.23 ± 2.19	62.85	56.34	62.72	0.1570
E1	61.69 ± 1.77	56.12 ± 1.74	60.74	55.47	61.96	0.1656
E4	61.69 ± 1.77	56.12 ± 1.74	–	–	–	–
E2	61.61 ± 1.99	58.34 ± 1.67	–	–	–	–
E6	61.58 ± 1.67	55.63 ± 2.45	–	–	–	–
E12	61.32 ± 1.81	55.97 ± 3.06	–	–	–	–
E7	60.81 ± 1.02	58.02 ± 0.82	–	–	–	–
E5	60.80 ± 2.13	54.51 ± 3.58	–	–	–	–

Table D.2: Java: stage-1 CV and available exploratory stage-2 test scores. F1/AP in percent; – denotes not evaluated in stage 2.

ID	CV micro F1 ± SD	CV macro F1 ± SD	Test micro F1	Test macro F1	Test macro AP	Test HL
E15	62.25 ± 4.16	64.95 ± 5.39	69.14	72.84	81.17	0.0933
E17	61.54 ± 2.59	65.06 ± 4.24	70.34	73.42	81.28	0.0921
E9	61.25 ± 5.55	64.69 ± 7.34	67.62	73.71	78.66	0.1068
E12	61.17 ± 5.19	64.97 ± 4.98	–	–	–	–
E6	61.15 ± 6.88	65.57 ± 7.53	–	–	–	–
E8	60.65 ± 5.21	63.61 ± 6.22	–	–	–	–
E5	60.55 ± 4.88	64.06 ± 4.10	–	–	–	–
E14	60.32 ± 2.05	64.39 ± 3.26	–	–	–	–
E13	60.01 ± 2.84	64.22 ± 4.11	–	–	–	–
E2	59.81 ± 5.93	64.93 ± 5.89	–	–	–	–
E11	59.80 ± 3.00	62.62 ± 4.88	–	–	–	–
E16	59.79 ± 4.04	62.86 ± 5.06	–	–	–	–
E1	59.68 ± 6.88	62.73 ± 7.53	69.47	69.46	77.08	0.0945
E4	59.68 ± 6.88	62.73 ± 7.53	–	–	–	–
E10	59.57 ± 4.62	63.24 ± 6.49	–	–	–	–
E3	59.54 ± 5.90	63.12 ± 6.86	68.39	71.11	81.14	0.1009
E7	59.18 ± 5.64	63.37 ± 5.17	–	–	–	–

Table D.3: Pharo: stage-1 CV and available exploratory stage-2 test scores. F1/AP in percent; – denotes not evaluated in stage 2.

ID	CV micro F1 ± SD	CV macro F1 ± SD	Test micro F1	Test macro F1	Test macro AP	Test HL
E3	71.52 ± 2.33	62.67 ± 3.72	69.19	60.15	67.42	0.0990
E9	71.24 ± 3.04	60.94 ± 4.64	71.57	60.58	66.93	0.0909
E8	71.04 ± 3.07	60.71 ± 4.07	–	–	–	–
E12	70.68 ± 3.67	62.45 ± 5.93	–	–	–	–
E1	70.44 ± 3.74	60.53 ± 5.17	73.71	67.20	66.81	0.0828
E4	70.44 ± 3.74	60.53 ± 5.17	–	–	–	–
E6	70.36 ± 2.78	62.76 ± 4.10	–	–	–	–
E17	69.93 ± 2.87	61.88 ± 3.14	70.80	62.83	70.63	0.0917
E5	69.92 ± 3.18	62.71 ± 3.90	–	–	–	–
E15	69.56 ± 4.50	58.57 ± 6.35	70.50	61.94	69.34	0.0917
E10	69.50 ± 2.53	57.61 ± 2.68	–	–	–	–
E2	69.12 ± 4.45	63.58 ± 4.47	–	–	–	–
E13	68.93 ± 3.47	58.60 ± 4.32	–	–	–	–
E11	68.05 ± 3.58	57.06 ± 6.66	–	–	–	–
E14	67.67 ± 2.82	56.75 ± 3.22	–	–	–	–
E16	67.48 ± 2.02	53.74 ± 3.49	–	–	–	–
E7	67.36 ± 2.08	61.60 ± 2.67	–	–	–	–

D.1 Execution Coverage and Numerical Validation

The three revised source notebooks were executed without hidden code replacements. Their analysis cells import a shared utility module for data validation and a consistent plotting style. Count matrices are computed in int64 before multiplication; every co-occurrence diagonal equals the corresponding positive support. Prepared targets and normalized texts match the canonical raw files. All five validation partitions cover every sentence exactly once, and identical texts do not cross training/evaluation boundaries within a split. Table D.4 records execution coverage. Figure files and source-cell mappings are retained in `results/EDA/manifest.json`.

Table D.4: Executed revised notebooks; every analysis cell completed successfully.

Notebook	Code cells	Figures	Errors
`EDA_all_datasets.ipynb`	12	20	0
`EDA_python_code_comment_classification.ipynb`	9	8	0
`EDA_visualizations.ipynb`	6	7	0

Color has a specified role in every chart. Language colors are fixed; within support plots, neutral gray denotes positive sentence counts and the language color denotes distinct positive text groups. Sequential blue maps encode a nonnegative magnitude. Diverging brown/blue maps are centered at zero and encode negative/positive prevalence differences, respectively. No color is used as an unsupported judgment of data or model quality.

D.2 Positive Support and Text Diversity

Annotation volume and text diversity are distinct. Multiple positive IDs may share identical text, while a text group can contain different targets. The bars quantify evidence diversity without asserting statistical independence. Figure D.1 records the observed distribution under the definitions above.

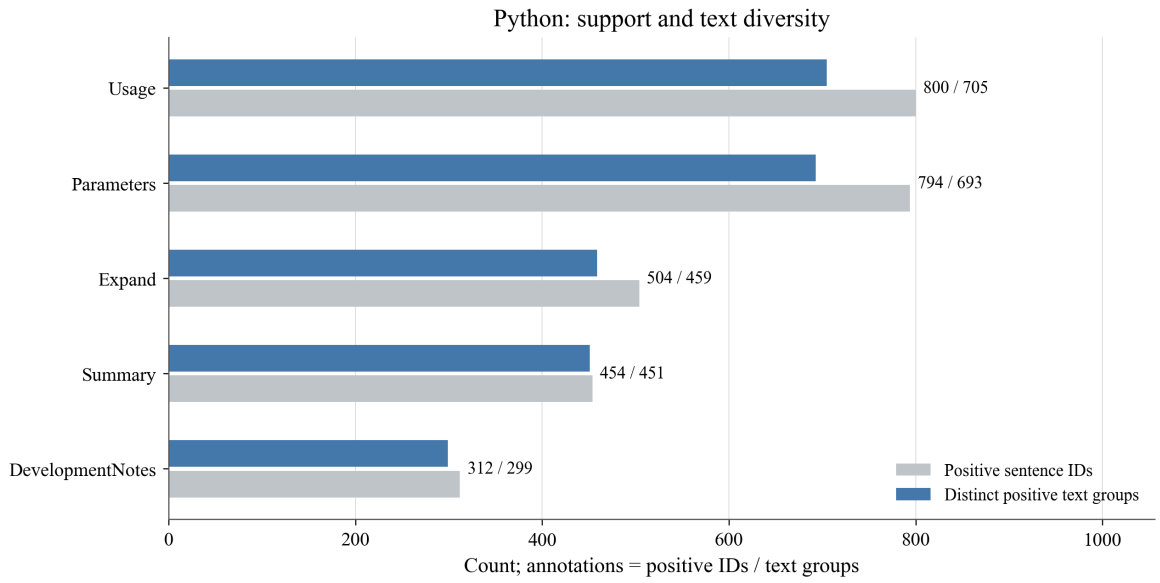


Figure D.1: Python: positive sentence IDs and distinct positive text groups; annotations report both counts.

Figure D.2 records the observed distribution under the definitions above.

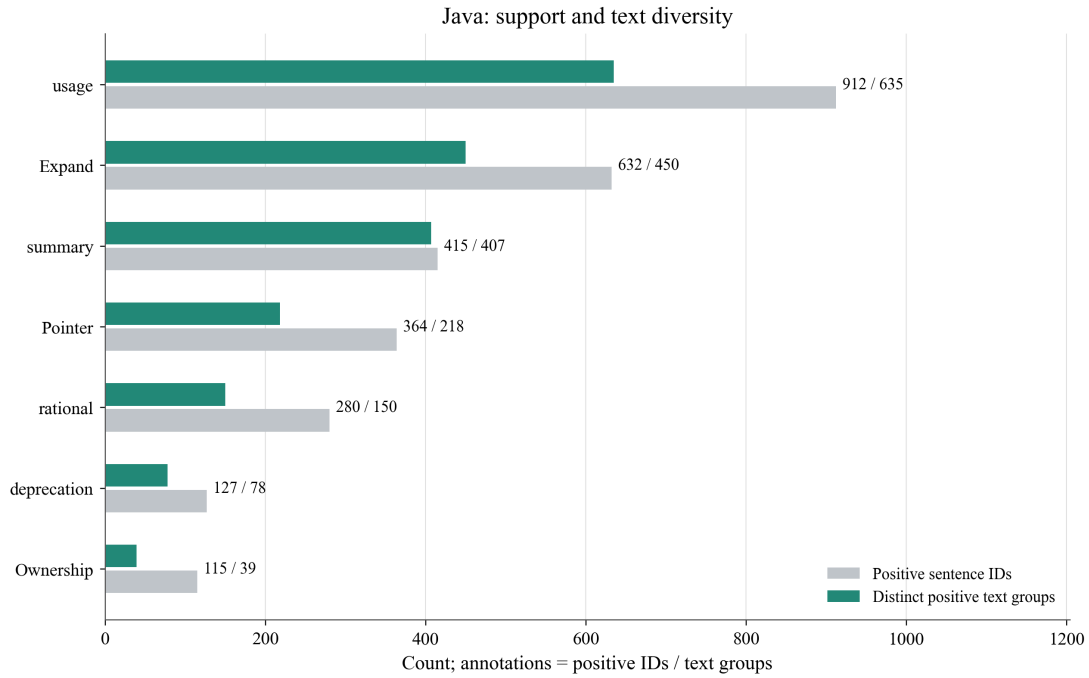


Figure D.2: Java: positive sentence IDs and distinct positive text groups; annotations report both counts.

Figure D.3 records the observed distribution under the definitions above.

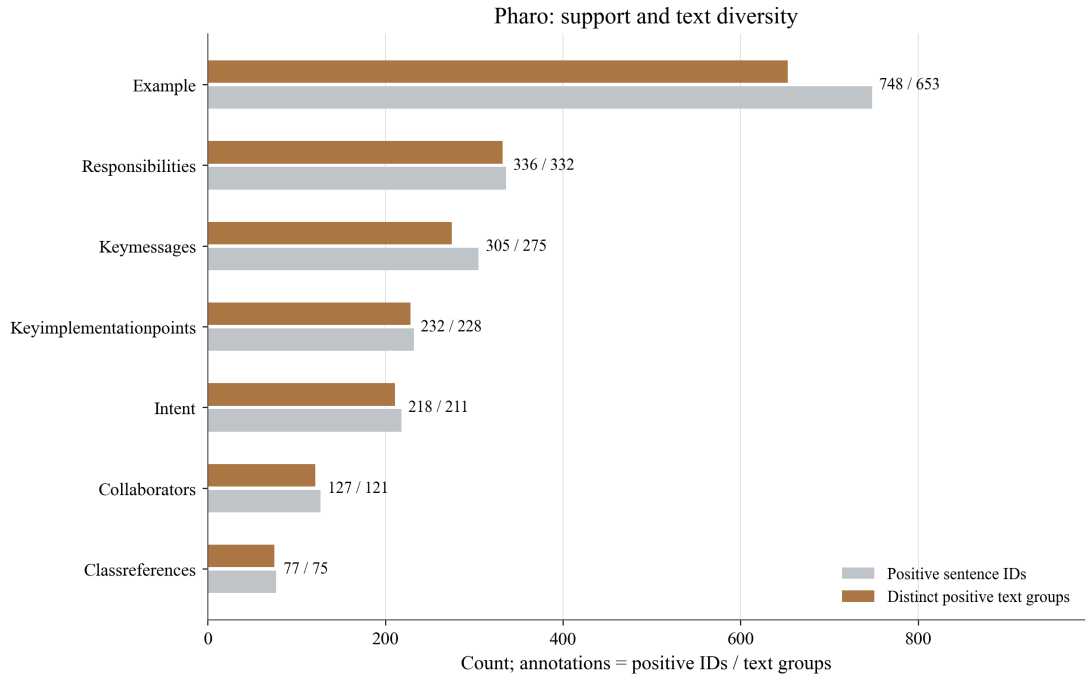


Figure D.3: Pharo: positive sentence IDs and distinct positive text groups; annotations report both counts.

D.3 Normalized Label Association

Jaccard similarity corrects raw intersections for union size. The same color range is used across all languages, and self-association is omitted. These descriptive associations do not establish causal dependence and are not the cosine-normalized training graph used by E9. Figure D.4 records the observed distribution under the definitions above.

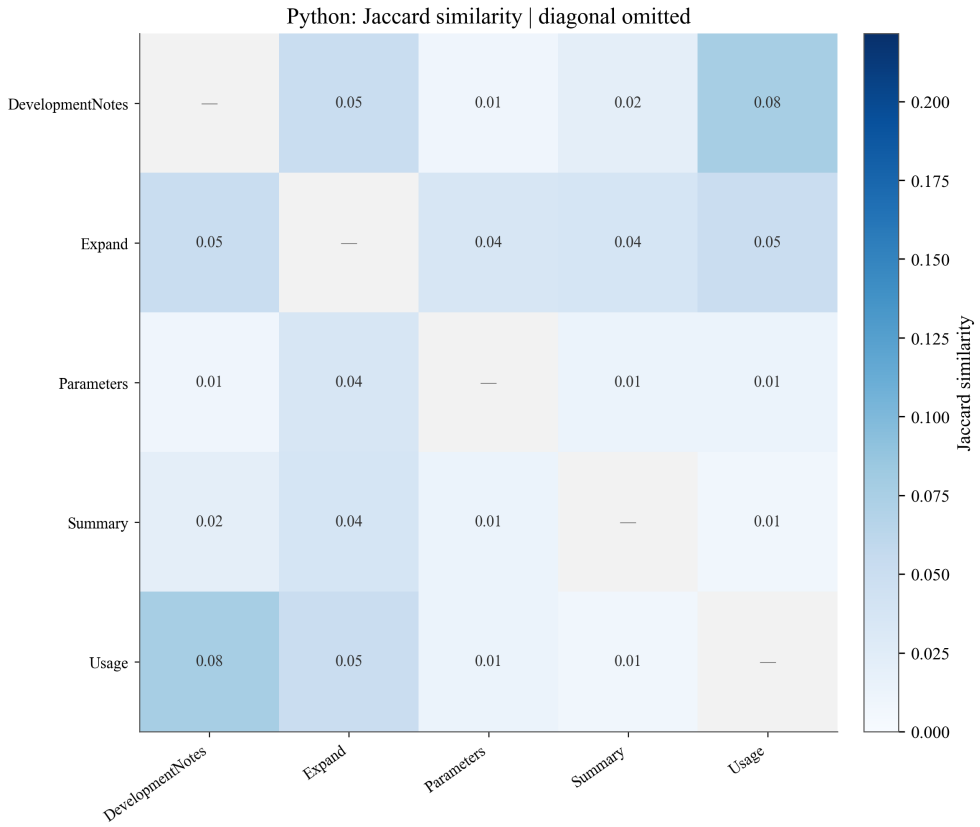


Figure D.4: Python: Jaccard similarity with a shared scale and omitted diagonal.

Figure D.5 records the observed distribution under the definitions above.

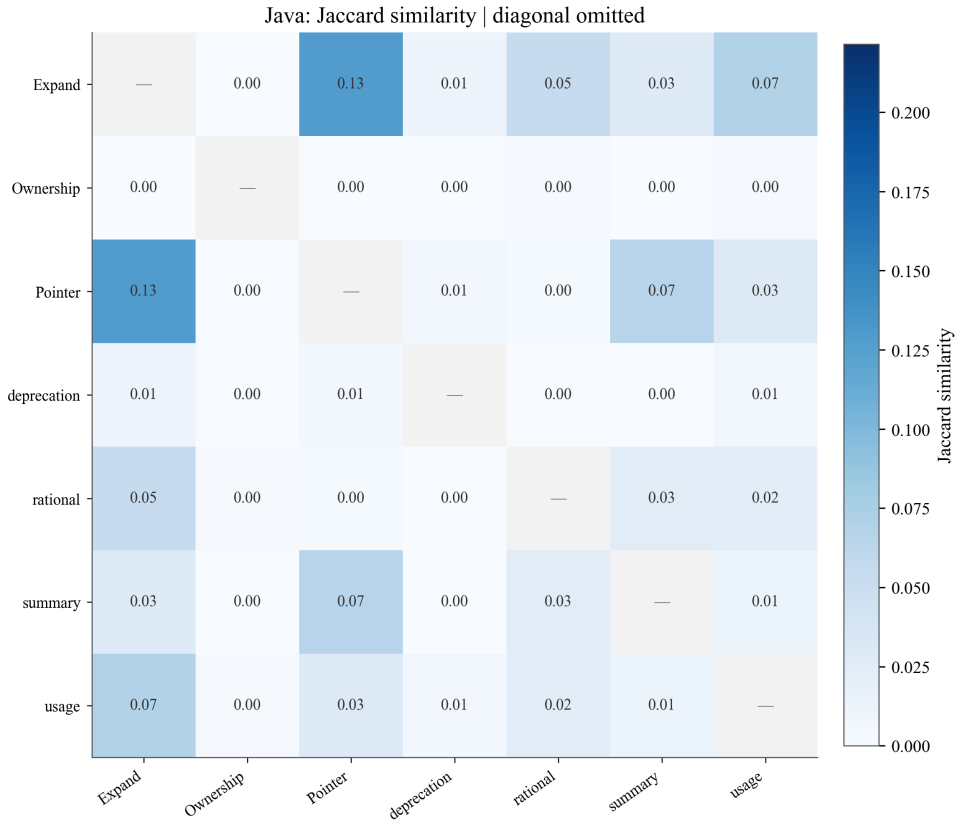


Figure D.5: Java: Jaccard similarity with a shared scale and omitted diagonal.

Figure D.6 records the observed distribution under the definitions above.

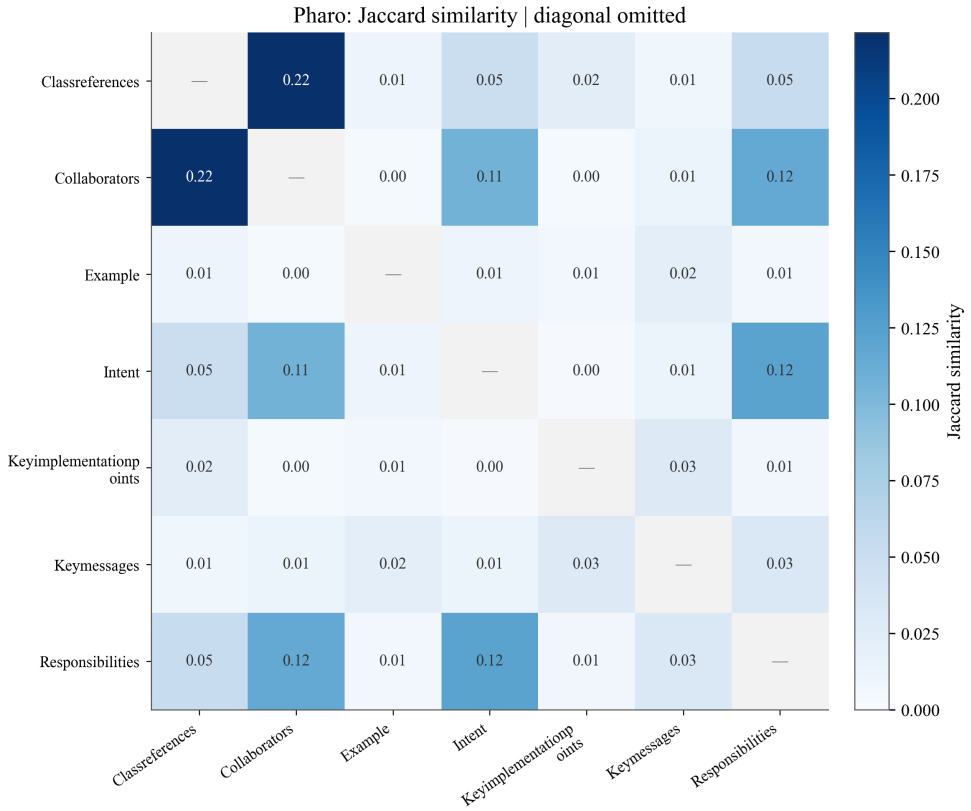


Figure D.6: Pharo: Jaccard similarity with a shared scale and omitted diagonal.

D.4 Class Concentration and Generalization Scope

Cumulative coverage ranks classes by sentence count. The diagonal is a uniform reference, not an optimal score. Shared class names across final training and test demonstrate that exact-text separation is weaker than class/project separation; the overlap fraction concerns class names, not leaked sentence IDs. Figure D.7 records the observed distribution under the definitions above.

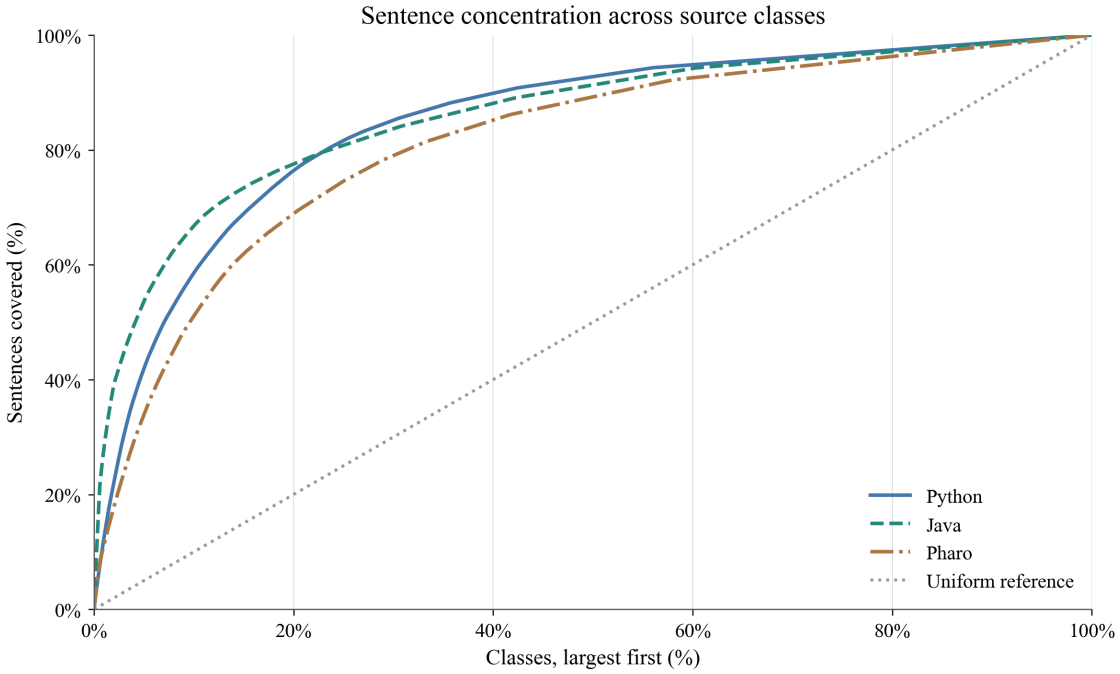


Figure D.7: Cumulative concentration of sentences across source classes.

Figure D.8 records the observed distribution under the definitions above.

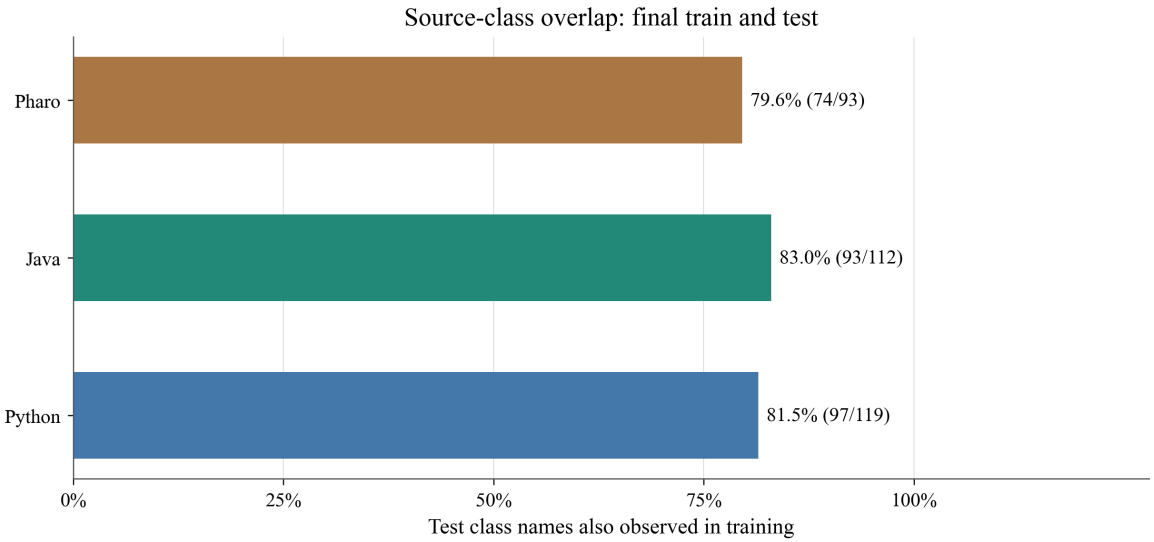


Figure D.8: Fraction of test class names also present in training, with count denominators.

D.5 Fixed Stage-2 Label Balance

Each point subtracts complete-population prevalence from the corresponding split prevalence. Positive and negative shifts use a common zero reference. Group indivisibility and integer sample counts constrain balance, especially for labels supported by few groups. These plots concern data composition, not test F1. Figure D.9 records the observed distribution under the definitions above.

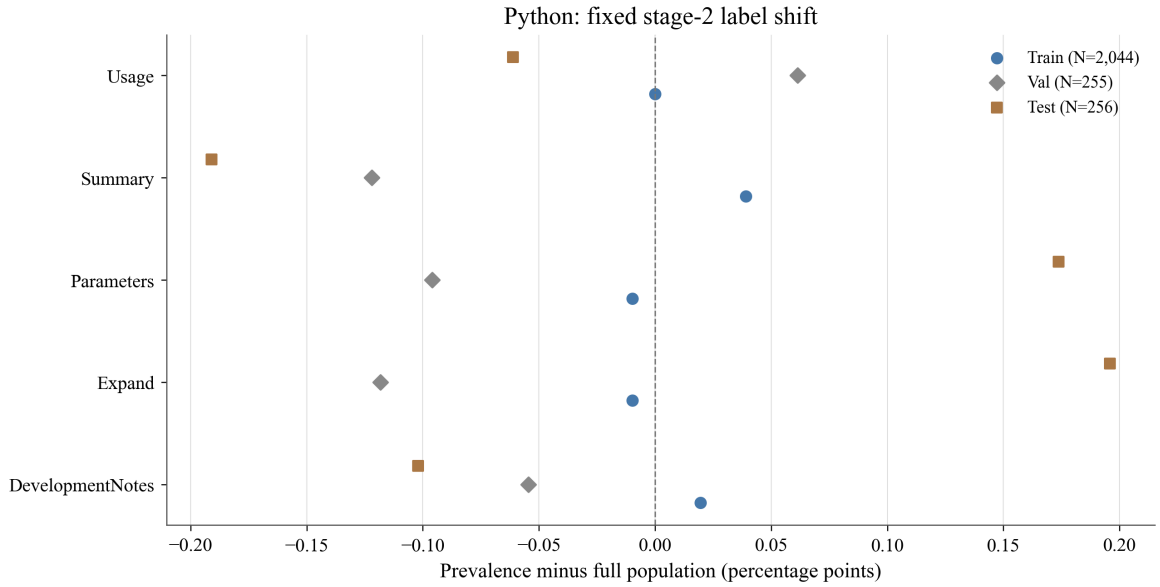


Figure D.9: Python: signed stage-2 prevalence shifts; legend identifies split and sample size.

Figure D.10 records the observed distribution under the definitions above.

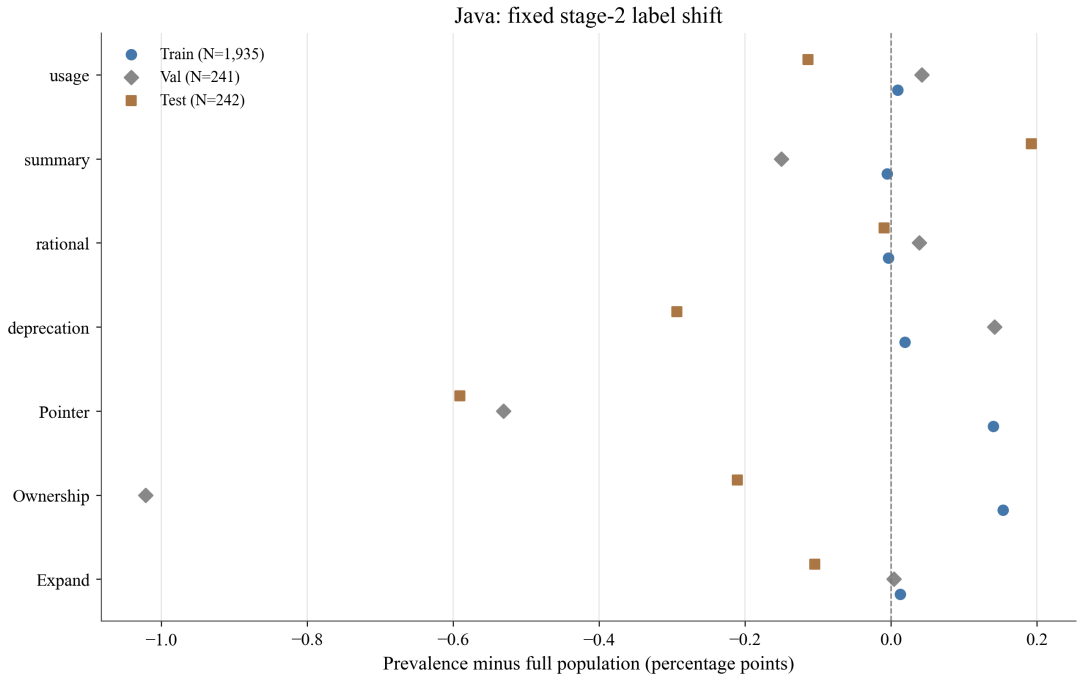


Figure D.10: Java: signed stage-2 prevalence shifts; legend identifies split and sample size.

Figure D.11 records the observed distribution under the definitions above.

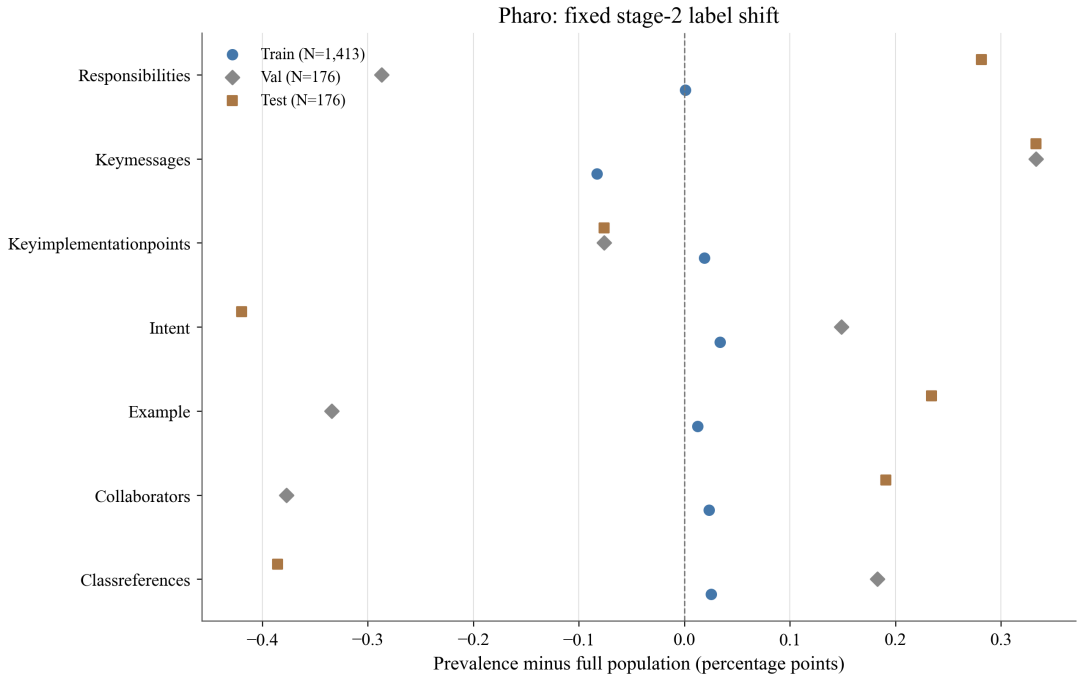


Figure D.11: Pharo: signed stage-2 prevalence shifts; legend identifies split and sample size.

D.6 Original Category-specific Partitions

The raw partition indicator belongs to an ID/category pair. These distributions are retained as dataset context and must not be identified with the present 80/10/10 split. A small original train/test prevalence gap does not establish independence of the subsequent exploratory evaluation. Figure D.12 records the observed distribution under the definitions above.

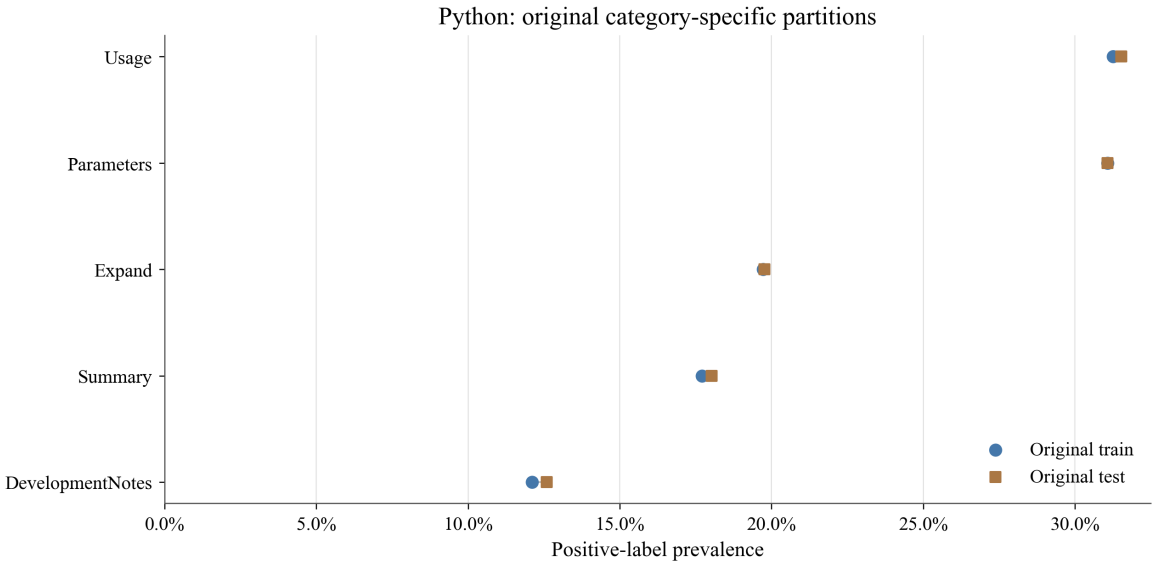


Figure D.12: Python: positive rates in original category-specific train/test partitions.

Figure D.13 records the observed distribution under the definitions above.

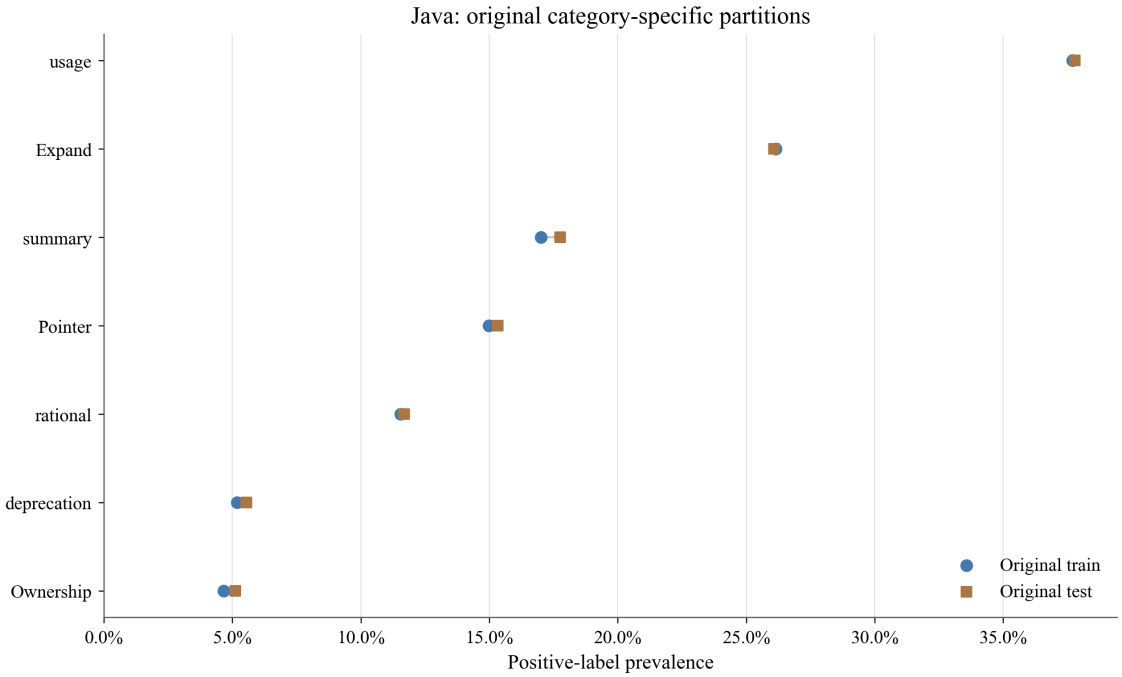


Figure D.13: Java: positive rates in original category-specific train/test partitions.

Figure D.14 records the observed distribution under the definitions above.

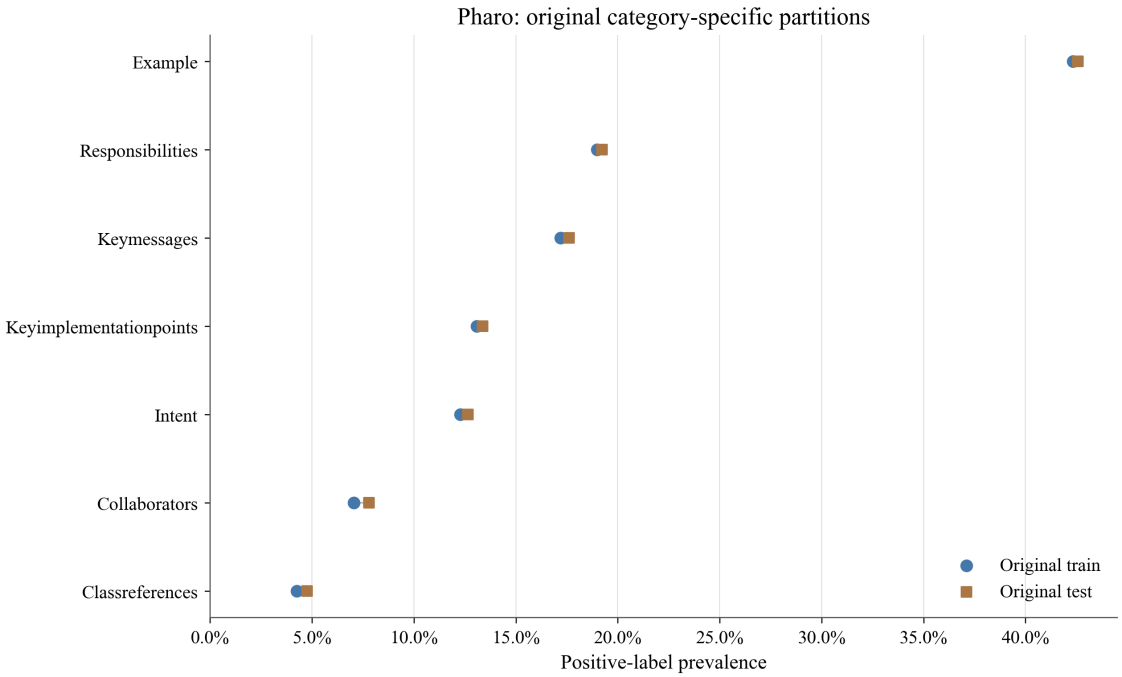


Figure D.14: Pharo: positive rates in original category-specific train/test partitions.

D.7 Python Conditional Relations and Group Constraints

Conditional co-occurrence reads rows as conditioning events. Its asymmetry is expected and does not indicate a prediction model. The largest-group contribution measures how strongly one text group constrains distribution of a label across folds. Neither quantity alone predicts model difficulty. Figure D.15 records the observed distribution under the definitions above.

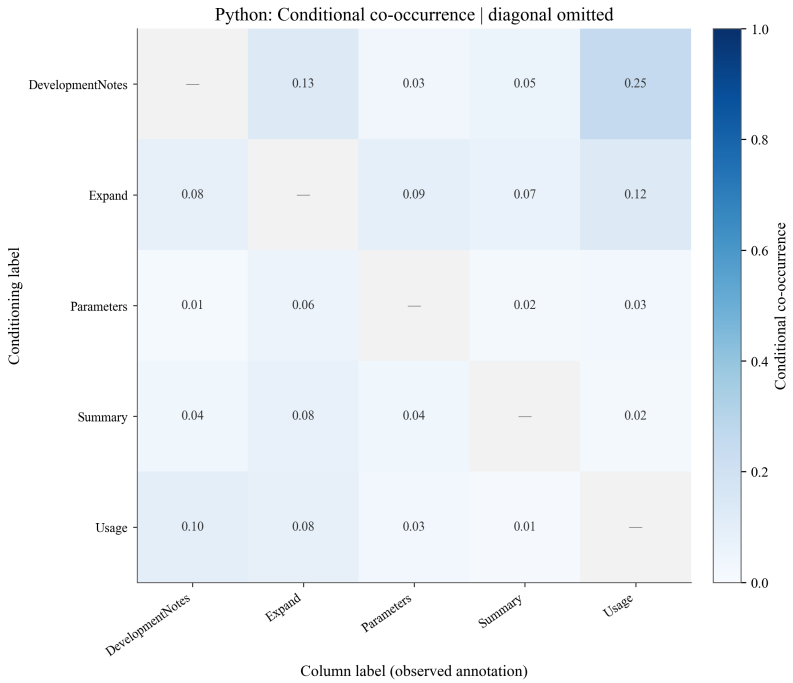


Figure D.15: Python conditional co-occurrence; row label is the conditioning event.

Figure D.16 records the observed distribution under the definitions above.

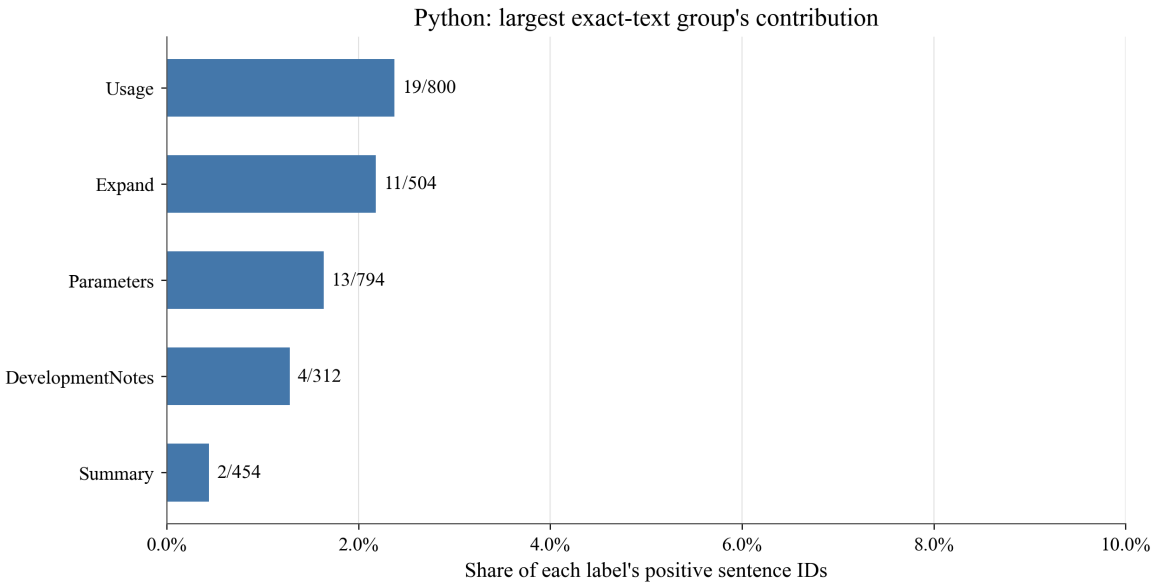


Figure D.16: Python positive-label support attributable to the largest exact-text group.

D.8 Reproducibility and Evidence Boundaries

The redesigned notebook set produces 35 PNG figures at 300 DPI. The present report selects population, tokenization, repetition, cardinality, grouping, association, and partition diagnostics rather than repeating every graph. Code cells retain executed tables and images. The exporter runs the source code without compatibility rewrites and replaces the old image set only after all notebooks complete successfully.

Label disagreement within identical text is reported as ambiguity requiring contextual review, not automatically as noise. Word length is kept distinct from tokenizer length. No significance tests, predictive error examples, calibration curves, or performance improvements are inferred from these descriptive analyses. The original 51 CV and 15 final runs remain the sole source of model scores. Comparable ML scores remain TBD. Confirmatory claims require a test population reserved before all selection.

The complete source-cell and figure mapping is available in the EDA manifest. The following raw-file digests identify the audited source population:

Python

`293914241158acbad61674b5321500bace54d42cb40e8800390cba7328ca4381`

Java

`82f1eb502eb2f0457865704d337192950812e5dbf2caaa63078fbc9ec7e5be28`

Pharo

`ddf4a52bcb69df8208b8dc5100827f2fe177c33849284baec93b739580aae560`